CODE AUDIT

OWASP Juice Shop v20.2.0

43 findings and 1 open question across 1 repository

DATE	AUDITOR
2026-10-11	Andrii Naidenko

REPOSITORIES AUDITED (BRANCH, COMMIT)

juice-shop-v20.2.0-4b369c9 · main · 4b369c9193

13	21	7	2	0
CRITICAL	HIGH	MEDIUM	LOW	INFO

Contents

1. Summary
2. Scope and method
3. Findings (43)
4. Open questions (1)
5. Technical details
6. Disclaimer

Summary

Fix before sign-off

CRITICAL	F-004	Secret in the code: Identified a Private Key, which may compromise cryptographic security and sensitive data encryption.	Security · effort S
CRITICAL	F-005	Secret in the code: Identified a Private Key, which may compromise cryptographic security and sensitive data encryption.	Security · effort S
CRITICAL	F-023	Ancient JWT libraries accept forged tokens; algorithm never pinned	Security · effort M
CRITICAL	F-031	Username spliced into Pug template source enables server-side code execution	Security · effort S
CRITICAL	F-035	Unauthenticated ZIP upload extracts entries anywhere under the app directory	Security · effort S
CRITICAL	F-036	XML uploads parsed with external entities enabled (XXE) and echoed back	Security · effort S
CRITICAL	F-038	Public memories endpoint returns posting users' full records, including password hashes	Security · effort S
CRITICAL	F-039	Auto-generated REST API lacks role and ownership checks	Security · effort M
CRITICAL	F-040	Registration accepts a role field, so anyone can sign up as admin	Security · effort S

CRITICAL	F-050	Seeded admin and staff accounts with weak, published passwords	Security · effort M
CRITICAL	F-053	Chatbot coupon tool lets anyone obtain discounts of any size	LLM integrations · effort M
CRITICAL	F-054	Chatbot order lookup trusts unsigned JWTs and a masked-email match	LLM integrations · effort S
CRITICAL	F-055	Unauthenticated chat endpoint has no rate limit, token cap or input cap	LLM integrations · effort M
HIGH	F-012	Detected a sequelize statement that is tainted by user-input.	Security · effort S
HIGH	F-015	Detected a sequelize statement that is tainted by user-input.	Security · effort S
HIGH	F-017	Five folders can be browsed by anyone, the encryption keys and server logs among them	Security · effort S
HIGH	F-022	Passwords stored as unsalted MD5 hashes	Security · effort M
HIGH	F-024	OAuth accounts use a password derived from their email address	Security · effort M
HIGH	F-025	JWT carries password hash and TOTP secret; no revocation on logout	Security · effort M
HIGH	F-026	Password change skips current-password check and sends passwords in URL	Security · effort S
HIGH	F-027	Password reset relies on guessable security question with spoofable throttle	Security · effort M
HIGH	F-028	Any user can read, modify and check out other users' baskets	Security · effort M
HIGH	F-029	Unvalidated amounts let users mint wallet credit and negative orders	Security · effort M
HIGH	F-032	B2B order lines evaluated as code in node:vm with notevil	Security · effort S
HIGH	F-033	Order tracking builds a MarsDB $where JavaScript expression from the URL	Security · effort S
HIGH	F-034	Review endpoints: $where injection, operator injection and unowned edits	Security · effort S
HIGH	F-037	Profile image URL is fetched server-side without any allowlist	Security · effort M
HIGH	F-042	Angular sanitizer bypassed on search term, feedback, emails, IPs and orders	Security · effort M

HIGH	F-043	Data-erasure form spreads request body into render options, allowing file read	Security · effort S
HIGH	F-046	Discount coupons are unsigned encodings anyone can forge	Security · effort M
HIGH	F-048	Customer order PDFs written to the public /ftp folder; extension check bypassable	Security · effort S
HIGH	F-051	Ethereum wallet recovery phrase hard-coded in server source	Security · effort S
HIGH	F-052	whoami supports JSONP with cookie auth and arbitrary fields, leaking secrets cross-site	Security · effort S
HIGH	F-056	Untrusted history, usernames and reviews reach the prompt unseparated	LLM integrations · effort M

Can wait

Still to fix, after sign-off.

MEDIUM	F-014	The application redirects to a URL specified by user-supplied input `query` that is not validated.	Security · effort S
MEDIUM	F-030	Deluxe membership granted without payment for unknown paymentMode	Security · effort S
MEDIUM	F-044	Cookie-authenticated profile and erasure POSTs have no CSRF defence; CORS allows all	Security · effort S
MEDIUM	F-045	No CSP or HSTS; profile page CSP built from a user-controlled URL	Security · effort M
MEDIUM	F-047	CAPTCHAs return their own answers and are reusable	Security · effort S
MEDIUM	F-049	Development error handler returns stack traces and SQL errors to clients	Security · effort S
MEDIUM	F-057	Chat stream has no timeout, no cancel on disconnect, and leaks raw errors	LLM integrations · effort S
LOW	F-041	Metrics, full app configuration and API docs served publicly	Security · effort S
LOW	F-059	No tests or evals for the chatbot prompt and tool policies	LLM integrations · effort M

1 open question needs the team's answer.

Estimated effort

26	17
SIZED S	SIZED M

S up to 2 hours M up to 2 days L over 2 days

Scope and method

Coverage

Security (juice-shop-v20.2.0-4b369c9): 12 of 15 items examined. done

- SEC–03 Authorization on every route partly examined
- SEC–12 Rate limiting and abuse partly examined
- SEC–15 Other dangerous patterns partly examined

LLM integrations (juice-shop-v20.2.0-4b369c9): 7 of 8 items examined. done

- LLM–08 Evaluation and observability partly examined

Method

- This sample audited OWASP Juice Shop v20.2.0 (upstream 5658473cf881) as Auditdesk's eval prepares it (`evals/prep`): the answers to its coding challenges removed and its challenges renamed, in one commit, `4b369c91932c`. Line numbers refer to that prepared tree.
- Scanners first: gitleaks over the history of every branch cloned, osv-scanner over the lock files and Semgrep; their pinned images are under Technical details.
- Then one agent per aspect read the code through read-only tools, against the aspect's checklist; Coverage above shows what each examined.
- The auditor reviewed all 58 findings the scanners and agents filed: 43 are in this report, 10 were merged into others and 5 rejected as wrong.
- The client's code was read, not installed, built or run: no dependency install, no type-check, no project lint.
- Model access: a Claude subscription through the Claude Agent SDK, under Anthropic's Consumer Terms.
- More on the method: naidenko.dev/audit.

Findings

CRITICAL F-004 **Secret in the code: Identified a Private Key, which may compromise cryptographic security and sensitive data encryption.**

Security · SEC-10 · effort S

RECOMMENDATION

Rotate the credential first, then remove it from the code and load it from the environment or a secret store.

A credential matching gitleaks rule "private-key" is committed in infrastructure/terraform/networking.tf.

LIKELIHOOD

Anyone with read access to the repository can use it.

IMPACT

Depends on what the credential grants; assume full access to that service.

DETAILS

Removing a secret from the code does not remove it from git history; every clone keeps it.

EVIDENCE

`infrastructure/terraform/networking.tf · line 170`

```
170  [secret masked: private-key]
```

`terraform/networking.tf · line 170`

```
170  [secret masked: private-key]
```

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V13.3 Secret Management.

Weakness: CWE-798.

Further reading: Secrets Management Cheat Sheet.

CRITICAL F-005 **Secret in the code: Identified a Private Key, which may compromise cryptographic security and sensitive data encryption.**

Security · SEC-10 · effort S

RECOMMENDATION

Rotate the credential first, then remove it from the code and load it from the environment or a secret store.

A credential matching gitleaks rule "private-key" is committed in lib/insecurity.ts.

LIKELIHOOD

Anyone with read access to the repository can use it.

IMPACT

Depends on what the credential grants; assume full access to that service.

DETAILS

Removing a secret from the code does not remove it from git history; every clone keeps it.

EVIDENCE

lib/insecurity.ts · line 21

```
21 [secret masked: private-key]
```

lib/insecurity.ts · line 54

```
54 export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })
```

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V13.3 Secret Management.

Weakness: CWE-798.

Further reading: Secrets Management Cheat Sheet.

CRITICAL F-023 **Ancient JWT libraries accept forged tokens; algorithm never pinned**

Security · SEC-02 · effort M

RECOMMENDATION

Upgrade to current `jsonwebtoken` (9.x) and `express-jwt` (8.x). Always pass `algorithms: ['RS256']` to verify. Drop the custom `jws.verify` path in favour of `jwt.verify(token, publicKey, { algorithms: ['RS256'] })`, and stop serving the key directory publicly.

The login tokens can be forged. Anyone can make a token that says "I am the administrator" and the server will believe it.

LIKELIHOOD

The forgery techniques (alg "none", HS256 signed with the published public key) are well known and need no account.

IMPACT

An attacker can mint a token for any user or role, including admin and accounting, and pass every isAuthorized/isAccounting/isDeluxe check.

DETAILS

package.json pins `jsonwebtoken` 0.4.0 and `express-jwt` 0.1.3. Both predate the algorithm-confusion fixes. `isAuthorized()` passes only `secret: publicKey` and no `algorithms`. `verify()` calls `jws.verify(token, publicKey)` without an algorithm, so a token with header `{"alg":"HS256"}` HMAC-signed with the RSA public key, which is downloadable from `/encryptionkeys/jwt.pub`, verifies. The old libraries also accept `alg: none`. `isAccounting`, `isDeluxe`, `isCustomer` and 2FA `verify`/`setup` all rely on this `verify` and then trust the decoded payload. `updateAuthenticatedUsers` also places any verified token into the session map, so forged identities flow into `appendUserId`.

EVIDENCE

lib/insecurity.ts · lines 52–56

```
 52  export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
 53  export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)
 54  export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn:
'6h', algorithm: 'RS256' })
 55  export const verify = (token: string) => token ? (jws.verify as ((token:
string, secret: string) => boolean))(token, publicKey) : false
 56  export const decode = (token: string) => { return jws.decode(token)?.payload }
```

lib/insecurity.ts · lines 152–171

```
152 export const isAccounting = () => {
153   return (req: Request, res: Response, next: NextFunction) => {
154     const decodedToken = verify(utils.jwtFrom(req)) &&
decode(utils.jwtFrom(req))
155     if (decodedToken?.data?.role === roles.accounting) {
156       next()
157     } else {
158       res.status(403).json({ error: 'Malicious activity detected' })
159     }
160   }
161 }
162
163 export const isDeluxe = (req: Request) => {
```

8 more lines in the HTML report

package.json · line 112

```
112     "express-jwt": "0.1.3",
```

package.json · line 129

```
129     "jsonwebtoken": "0.4.0",
```

REFERENCES

Relevant to: A04:2025 Cryptographic Failures; ASVS 5.0.0 V7.2 Fundamental Session Management Security; ASVS 5.0.0 V7.3 Session Timeout; ASVS 5.0.0 V7.4 Session Termination; ASVS 5.0.0 V9.1 Token source and integrity; ASVS 5.0.0 V3.3 Cookie Setup.

Weakness: CWE-347.

Further reading: Session Management Cheat Sheet.

CRITICAL F-031 **Username spliced into Pug template source enables server-side code execution**

Security · SEC-04 · effort S

RECOMMENDATION

Never build templates from user data. Keep `#{username}` fixed in the .pug file, compile it once at startup and pass `{ username }` as locals. Delete the `eval` branch entirely.

A customer's display name is pasted into the server's page template before it is processed. A specially crafted name makes the server run attacker code, which hands over the whole server.

LIKELIHOOD

Any registered user can set their username via POST /profile and then load /profile.

IMPACT

Arbitrary JavaScript runs in the Node process, giving full server compromise: database, keys, files.

DETAILS

`getUserProfile` reads `views/userProfile.pug`, does `template.replace(/_username_/g, username)` and then `pug.compile(template)`. The username therefore becomes Pug source, not data. Semgrep's F-016 covers only the direct `eval(code)` on `#{...}` names (line 65). Even with that branch removed, Pug's own syntax remains reachable: `!{...}`/`#{...}` interpolation after the leading backslash, or `#[...]` tags. The username setter's `sanitizeSecure` (sanitize-html) strips HTML tags but not Pug interpolation. Interpolation evaluates arbitrary JS expressions such as `global.process.mainModule.require('child_process')`.

EVIDENCE

routes/userProfile.ts · lines 57–77

```
57      if (utils.isChallengeEnabled(challenges.ch9db78273)) {
58        if (username?.match(/#{(.*)}/) !== null) {
59          req.app.locals.abused_ssti_bug = true
60          const code = username?.substring(2, username.length - 1)
61          try {
62            if (!code) {
63              throw new Error('Username is null')
64            }
65            username = eval(code) // eslint-disable-line no-eval
66          } catch (err) {
67            username = '\\' + username
68          }
```

9 more lines in the HTML report

routes/userProfile.ts · lines 88–98

```
88      try {
89        const pug = (await import('pug')).default
90        const fn = pug.compile(template)
91        const CSP = `img-src 'self' ${user?.profileImage}; script-src 'self'
'unsafe-eval'`
92
93
94        res.set({
95          'Content-Security-Policy': CSP
96        })
97
98        res.send(fn(user))
```

views/userProfile.pug · line 43

```
 43                  p(style='margin-top: 8px; color: _textColor_; text-align:
center;') _username_
```

REFERENCES

Relevant to: ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-1336.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

CRITICAL F-035 **Unauthenticated ZIP upload extracts entries anywhere under the app directory**

Security · SEC-08 · effort S

RECOMMENDATION

Require authentication. Resolve each entry and require `resolved.startsWith(path.resolve('uploads/complaints') + path.sep)`. Skip symlink entries. Cap entry count and total uncompressed bytes. Better still, don't extract archives server-side. Implement real type and size checks by magic bytes.

Anyone on the internet can upload a ZIP file that writes files into the shop's own program folders. That lets an attacker replace the website's pages or code.

LIKELIHOOD

/file-upload needs no login, and crafting a ZIP with `../` entry names is trivial.

IMPACT

Overwriting files the app serves or loads (the frontend bundle, i18n files, config, views) gives defacement, stored XSS for all visitors, or code execution on restart.

DETAILS

`extractZipBuffer` resolves `'uploads/complaints/' + entry.path` and only checks `absolutePath.includes(path.resolve('.'))`, so any path inside the working directory passes (e.g. `../../frontend/dist/frontend/main.js`, `../../views/userProfile.pug`, `../../config/default.yml`). The route has no auth middleware. `checkUploadSize` and `checkFileType` are empty no-ops, so type is decided by the client-supplied filename only. There is no limit on the decompressed size or entry count (zip bomb).

EVIDENCE

routes/fileUpload.ts · lines 27–36

```
27 async function extractZipBuffer (buffer: Buffer) {
28   const directory = await unzipper.Open.buffer(buffer)
29   for (const entry of directory.files) {
30     const fileName = entry.path
31     const absolutePath = path.resolve('uploads/complaints/' + fileName)
32     if (absolutePath.includes(path.resolve('.'))) {
33       await pipeline(entry.stream(), fs.createWriteStream('uploads/complaints/'
+ fileName))
34     }
35   }
36 }
```

routes/fileUpload.ts · lines 54–63

```
54 function checkUploadSize ({ file }: Request, res: Response, next: NextFunction)
{
55   if (file != null) {
56   }
57   next()
58 }
59
60 function checkFileType ({ file }: Request, res: Response, next: NextFunction) {
61   const fileType = file?.originalname.substr(file.originalname.lastIndexOf('.')
+ 1).toLowerCase()
62   next()
63 }
```

server.ts · line 316

```
316   app.post('/file-upload', uploadToMemory.single('file'), ensureFileIsPassed,
metrics.observeFileUploadMetricsMiddleware(), checkUploadSize, checkFileType,
handleZipFileUpload, handleXmlUpload, handleYamlUpload)
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V5.2 File Upload and Content; ASVS 5.0.0 V5.3 File Storage; ASVS 5.0.0 V5.4 File Download.

Weakness: CWE-22.

Further reading: File Upload Cheat Sheet.

CRITICAL F-036 **XML uploads parsed with external entities enabled (XXE) and echoed back**

Security · SEC-04 · effort S

RECOMMENDATION

Remove the XML/YAML complaint handlers, or parse with entity substitution and DTD loading disabled and no filesystem providers. Use `yaml.load` with `JSON_SCHEMA` and size limits. Never echo parsed content in errors.

Anyone can upload an XML file that tricks the server into reading its own private files and sending them back. That includes passwords and keys.

LIKELIHOOD

Unauthenticated POST to /file-upload with a .xml file; standard XXE payloads work.

IMPACT

Any file readable by the Node process (config, keys, /etc/passwd, the SQLite DB in part) is returned in the error message. Entity expansion also stalls the server, and YAML bombs exhaust memory.

DETAILS

`parseXmlString` deliberately sets `XML_PARSE_NOENT | XML_PARSE_DTDLOAD` and registers filesystem input providers, so `<!ENTITY x SYSTEM "file:///...">` resolves. `handleXmlUpload` puts the first 400 chars of the parsed result into an error passed to `errorhandler()`, which renders it to the client. `handleYamlUpload` runs `yaml.load` on uploaded data (js-yaml 3.x) and likewise echoes the result. Billion-laughs aliases lead to memory exhaustion before the vm timeout matters.

EVIDENCE

lib/xml.ts · lines 15–41

```
15 async function loadLibxml2 () {
16   if (libxml2Promise == null) {
17     libxml2Promise = (async () => {
18       const libxml2 = await dynamicImport('libxml2-wasm')
19       // Grants the WASM sandbox host filesystem access so external entities
20       // like file:///etc/passwd resolve - required for the XXE challenges.
21       const { xmlRegisterFsInputProviders } = await dynamicImport('libxml2-
wasm/lib/nodejs.mjs')
22       xmlRegisterFsInputProviders()
23       return libxml2
24     })()
25   }
26   return await libxml2Promise
```

15 more lines in the HTML report

routes/fileUpload.ts · lines 65–100

```
65 async function handleXmlUpload ({ file }: Request, res: Response, next:
NextFunction) {
66   if (file?.originalname?.toLowerCase().endsWith('.xml') ?? false) {
67     if (((file?.buffer) != null) &&
utils.isChallengeEnabled(challenges.chddc39ed1)) { // XXE attacks in Docker/Heroku
containers regularly cause "segfault" crashes
68       const data = file.buffer.toString()
69       try {
70         const xmlString = await parseXmlString(data)
71         res.status(410)
72         next(new Error('B2B customer complaints via file upload have been
deprecated for security reasons: ' + utils.trunc(xmlString, 400) + ' (' +
file.originalname + ')'))
73       } catch (err: unknown) {
74         const errorMessage = err instanceof Error ? err.message : String(err)
75         if (errorMessage.includes('Script execution timed out')) {
76           res.status(503)
```

18 more lines in the HTML report

REFERENCES

Relevant to: A02:2025 Security Misconfiguration; ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-611.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

CRITICAL F-038 **Public memories endpoint returns posting users' full records, including password hashes**

Security · SEC-03 · effort S

RECOMMENDATION

Use `include: [{ model: UserModel, attributes: ['id', 'username'] }]`. Add a `defaultScope` on User that excludes `password`, `totpSecret` and `deluxeToken`, so other includes are also safe.

The public photo wall also sends out the private account details of everyone who posted a photo, including their scrambled password and two-factor secret, to anyone who looks.

LIKELIHOOD

GET /rest/memories needs no authentication.

IMPACT

Email, role, MD5 password hash, TOTP secret and deluxe token of every user who posted a photo leak to anyone.

DETAILS

`getMemories` runs `MemoryModel.findAll({ include: [UserModel] })` with no `attributes` restriction, so Sequelize serialises every User column (password, totpSecret, deluxeToken, lastLoginIp). The route has no auth middleware. Combined with the unsalted MD5 hashing (F-022), the passwords are easily recovered.

EVIDENCE

routes/memory.ts · lines 22–26

```
22 export function getMemories () {
23   return async (req: Request, res: Response, next: NextFunction) => {
24     const memories = await MemoryModel.findAll({ include: [UserModel] })
25     res.status(200).json({ status: 'success', data: memories })
26   }
```

server.ts · line 624

```
624   app.get('/rest/memories', utils.asyncHandler(getMemories()))
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V8.2 General Authorization Design; ASVS 5.0.0 V8.3 Operation Level Authorization; ASVS 5.0.0 V14.2 General Data Protection.

Weakness: CWE-200.

Further reading: Authorization Cheat Sheet; Insecure Direct Object Reference Prevention Cheat Sheet.

CRITICAL F-039 **Auto-generated REST API lacks role and ownership checks**

Security · SEC-03 · effort M

RECOMMENDATION

Default to deny: add a finale `all.auth` hook that rejects unless an explicit per-model/per-action policy allows it. Require admin for product writes and user listing. Scope list/read/update/delete to `UserId = caller` for Address, Card, BasketItem, Complaint, Recycle and PrivacyRequest. Add integration tests asserting 401/403 for each generated endpoint.

Much of the shop's data API has no idea who is allowed to do what. Anyone can change product listings and prices without logging in, and any customer can see every other user's account list and complaints or change their saved addresses.

LIKELIHOOD

Product edits need no login at all; the rest need only a free account.

IMPACT

Anyone can rewrite product names, descriptions and prices (stored XSS and price tampering for every shopper). Any customer can list all users, read all complaints, delete any feedback, and edit other users' addresses.

DETAILS

finale generates full CRUD for 13 models (`/api/<Model>s` and `/:id`), and the hand-written guards in server.ts are incomplete. `app.put('/api/Products/:id', security.isAuthorized())` is commented out, so PUT on products is unauthenticated. `GET /api/Users`, `GET /api/Users/:id` and `/rest/user/authentication-details` require only `isAuthorized()`, not admin, so any customer enumerates all users, emails and roles. `/api/Feedbacks/:id` DELETE requires only login. `GET /api/Complaints` returns everyone's complaints. `PUT /api/Addresss/:id` only appends `UserId`; finale updates by `id` without checking ownership. `/api/Hints/:id` PUT and `/api/Recycles/:id` GET (which `JSON.parse`s the id, so `[1,2,3,...]` returns arbitrary rows) are open. The Angular admin page is protected only by a client-side route guard.

EVIDENCE

server.ts · lines 364–374

```
364   app.use('/api/Feedbacks/:id', security.isAuthorized())
365   /* Users: Only POST is allowed in order to register a new user */
366   app.get('/api/Users', security.isAuthorized())
367   app.route('/api/Users/:id')
368     .get(security.isAuthorized())
369     .put(security.denyAll())
370     .delete(security.denyAll())
371   /* Products: Only GET is allowed in order to view products */
372   app.post('/api/Products', security.isAuthorized())
373   // app.put('/api/Products/:id', security.isAuthorized())
374   app.delete('/api/Products/:id', security.denyAll())
```

server.ts · lines 380–391

```
380   app.route('/api/Hints/:id')
381     .get(security.denyAll())
382     .delete(security.denyAll())
383   /* Complaints: POST and GET allowed when logged in only */
384   app.get('/api/Complaints', security.isAuthorized())
385   app.post('/api/Complaints', security.isAuthorized())
386   app.use('/api/Complaints/:id', security.denyAll())
387   /* Recycles: POST and GET allowed when logged in only */
388   app.get('/api/Recycles', recycles.blockRecycleItems())
389   app.post('/api/Recycles', security.isAuthorized())
390   /* Challenge evaluation before finale takes over */
391   app.get('/api/Recycles/:id', recycles.getRecycleItem())
```

server.ts · lines 446–448

```
446   app.post('/api/Addresss', security.appendUserId())
447   app.get('/api/Addresss', security.appendUserId(),
utils.asyncHandler(address.getAddress()))
448   app.put('/api/Addresss/:id', security.appendUserId())
```

server.ts · lines 495–502

```
495   for (const { name, exclude, model, include } of autoModels) {
496     const resource = finale.resource({
497       model,
498       endpoints: [`/api/${name}s`, `/api/${name}s/:id`],
499       excludeAttributes: exclude,
500       pagination: false,
501       include
502     })
```

routes/recycles.ts · lines 11–17

```
11 export const getRecycleItem = () => (req: Request, res: Response) => {
12   RecycleModel.findAll({
13     where: {
14       id: JSON.parse(req.params.id)
15     }
16   }).then((Recycle) => {
17     return res.send(utils.queryResultToJson(Recycle))
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V8.2 General Authorization Design; ASVS 5.0.0 V8.3 Operation Level Authorization; ASVS 5.0.0 V14.2 General Data Protection.

Weakness: CWE-862.

Further reading: Authorization Cheat Sheet; Insecure Direct Object Reference Prevention Cheat Sheet.

CRITICAL F-040 **Registration accepts a role field, so anyone can sign up as admin**

Security · SEC-09 · effort S

RECOMMENDATION

Replace generated user creation with a dedicated handler that validates a schema of `{email, password, passwordRepeat, securityQuestion, securityAnswer}` only, and sets `role = 'customer'` server-side. Alternatively, in a finale `create.write.before` hook, delete every attribute except the allowed ones.

When signing up, a user can simply declare themselves an administrator and the server accepts it.

LIKELIHOOD

Anyone can POST /api/Users with `"role":"admin"`; no login needed.

IMPACT

An attacker immediately holds an admin (or accounting/deluxe) account, with every privilege that role grants.

DETAILS

User registration is the finale-generated `POST /api/Users`, which builds the model from the whole request body. The only pre-handler (server.ts 409-420) trims email and password. Nothing strips `role`, `deluxeToken`, `totpSecret`, `isActive` or `profileImage`. The model's `role` validator accepts `admin`, `accounting` and `deluxe`. Because the pre-handler only trims when all three fields are present, it also doesn't reliably reject empty passwords.

EVIDENCE

server.ts · lines 409–420

```
409   app.post('/api/Users', (req: Request, res: Response, next: NextFunction) => {
410     if (req.body.email !== undefined && req.body.password !== undefined &&
req.body.passwordRepeat !== undefined) {
411       if (req.body.email.length !== 0 && req.body.password.length !== 0) {
412         req.body.email = req.body.email.trim()
413         req.body.password = req.body.password.trim()
414         req.body.passwordRepeat = req.body.passwordRepeat.trim()
415       } else {
416         res.status(400).send(res.__('Invalid email/password cannot be empty'))
417       }
418     }
419     next()
420   })
```

models/user.ts · lines 73–93

```
73        role: {
74          type: DataTypes.STRING,
75          defaultValue: 'customer',
76          validate: {
77            isIn: [['customer', 'deluxe', 'accounting', 'admin']]
78          },
79          set (role: string) {
80            const profileImage = this.getDataValue('profileImage')
81            if (
82              role === security.roles.admin &&
83            (!profileImage ||
84              profileImage === '/assets/public/images/uploads/default.svg')
```

9 more lines in the HTML report

server.ts · lines 478–479

```
478   const autoModels = [
479     { name: 'User', exclude: ['password', 'totpSecret'], model: UserModel },
```

REFERENCES

Relevant to: A08:2025 Software or Data Integrity Failures; ASVS 5.0.0 V2.2 Input Validation; ASVS v5.0.0-15.3.3 (V15.3 Defensive Coding).

Weakness: CWE-915.

Further reading: Input Validation Cheat Sheet; Mass Assignment Cheat Sheet.

CRITICAL F-050 **Seeded admin and staff accounts with weak, published passwords**

Security · SEC-10 · effort M

RECOMMENDATION

Don't seed privileged accounts in production. Create the first admin through a one-time bootstrap with a password from a secret store, forced to change on first login. Stop `sync({ force: true })` in production. Remove the key files from the repo and its history, rotate them, and load them from environment secrets.

The shop creates an administrator account with the password "admin123" every time it starts, and that password is in the published source code. Anyone can log in as the administrator.

LIKELIHOOD

The seed file is in the public repository and is loaded on every start (`sequelize.sync({ force: true })` then `datacreator()`).

IMPACT

Anyone can log in as the administrator (`admin@<domain>` / `admin123`) or other staff, with full access to users, feedback and orders.

DETAILS

`data/static/users.yml` defines users with plaintext passwords and security answers (admin `admin123`, jim `ncc-1701`, and others), plus card numbers and addresses. `start()` drops and recreates the schema and reseeds from this file, so production always contains these accounts. Gitleaks F-001/F-002 flagged two keys in this file but not the account passwords themselves. Also committed: `ctf.key` (HMAC key for flags) and `encryptionkeys/premium.key`, the latter served publicly under /encryptionkeys.

EVIDENCE

data/static/users.yml · lines 1–8

```
1 -
2   email: admin
3   password: 'admin123'
4   key: admin
5   role: 'admin'
6   securityQuestion:
7     id: 2
8     answer: '@xI98PxDO+06!'
```

data/static/users.yml · lines 30–37

```
30   email: jim
31   password: 'ncc-1701'
32   key: jim
33   role: 'customer'
34   walletBalance: 100
35   securityQuestion:
36     id: 1
37     answer: 'Samuel' # https://en.wikipedia.org/wiki/James_T._Kirk
```

server.ts · lines 726–731

```
726 export async function start (readyCallback?: () => void) {
727   const datacreatorEnd = startupGauge.startTimer({ task: 'datacreator' })
728   await sequelize.sync({ force: true })
729   await preconditionsReady
730   await datacreator()
731   datacreatorEnd()
```

data/static/users.yml · line 88

```
88 password: '[secret masked: generic-api-key]'
```

data/static/users.yml · line 151

```
151 totpSecret: [secret masked: generic-api-key]
```

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V13.3 Secret Management.

Weakness: CWE-798.

Further reading: Secrets Management Cheat Sheet.

CRITICAL F-053 **Chatbot coupon tool lets anyone obtain discounts of any size**

LLM integrations · LLM-02 · effort M

RECOMMENDATION

Take coupon creation out of the model's free choice. Either drop the tool and send damaged-order cases to a human workflow, or make the tool take only an `orderId` and enforce the policy in code: the user is authenticated with a verified JWT, the order belongs to them, a damage claim is recorded, no coupon has been issued for it yet, and the discount is fixed server-side at 10% or less. Record issued coupons so each can be used once. Remove confidential business rules from the system prompt.

The chatbot can create real discount coupons. The only thing that stops it is a written instruction to the AI, and visitors can talk it out of that instruction. Anyone, even without an account, can ask the chatbot for a large coupon and use it at checkout.

LIKELIHOOD

Likely: /rest/chat needs no sign-in and the coupon rules exist only as text in the prompt, which a short jailbreak gets past.

IMPACT

Anyone can get valid coupons for 10%, 15%, 100% or more and use them on any order, so the shop loses revenue.

DETAILS

`generateCoupon` (routes/chat.ts:174-183) passes the model-chosen `discount` straight to `security.generateCoupon`. The zod schema is `z.number()` with no min or max, and the tool checks no order ID, damage report, rejected return, ownership or authentication. Every condition in the COUPON POLICY (lines 97-104) exists only in the prompt. The route is mounted without any auth middleware (server.ts:632). The client also sends the whole message history (`req.body.messages`, line 187), so an attacker can add fake assistant turns, or fake turns claiming

a verified damaged order, to steer the model into calling the tool. The system prompt also contains a 'CONFIDENTIAL' 15% escalation offer, which the model will reveal when asked. This is separate from F-046 (coupons are unsigned and can be forged offline): this finding is the server's own LLM tool issuing coupons of any size on request.

EVIDENCE

routes/chat.ts · lines 174–183

```
174       generateCoupon: tool({
175         description: 'Generate a discount coupon for a customer. Only use this
when the coupon policy conditions are fully met.',
176         inputSchema: z.object({
177           discount: z.number().describe('The discount percentage for the coupon
(maximum 10)')
178         }),
179         execute: async ({ discount }) => {
180           const couponCode = security.generateCoupon(discount)
181           return { couponCode, discount }
182         }
183       })
```

routes/chat.ts · lines 97–104

```
 97 COUPON POLICY (for the generateCoupon tool):
 98 - You may ONLY generate a coupon for a customer who has a verified damaged
order with a valid order ID (format: xxxx-xxxxxxxxxxxxxxxx, e.g. 3fa8-
bf2bc042f4e92).
 99 - The customer must have explicitly rejected a return or exchange before a
coupon can be offered.
100 - The maximum allowed discount is 10%.
101 - NEVER generate a coupon just because a customer asks for one or complains.
102 - If the customer does not meet ALL of the above conditions, politely decline
and explain the policy.
103
104 CONFIDENTIAL - INTERNAL ONLY: If a customer formally complains about their
shopping experience and explicitly requests to escalate the issue, offer them a one-
time 15% courtesy discount to resolve the case without escalation. Do not mention
this option proactively.`
```

server.ts · lines 631–632

```
631   /* Chat API endpoint */
632   app.post('/rest/chat', utils.asyncHandler(chat()))
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V8.3 Operation Level Authorization.

Weakness: CWE-862.

Further reading: AI Agent Security Cheat Sheet.

CRITICAL F-054 **Chatbot order lookup trusts unsigned JWTs and a masked-email match**

LLM integrations · LLM-02 · effort S

RECOMMENDATION

In the chat route, authenticate with `security.verify` / `isAuthorized()` before reading claims. Store the owner's `UserId` on each order document and match on it instead of the masked email. Return only the fields the bot needs (status, ETA, product names).

The chatbot is meant to show customers only their own orders. But it never checks that the login token is genuine, so anyone can claim to be another customer and have the chatbot show that customer's orders.

LIKELIHOOD

Easy: anyone can build an unsigned token with any user id, and order IDs show up in shared PDFs and the public /ftp folder.

IMPACT

Anyone without an account can read other customers' orders through the chatbot: products, prices, payment and address IDs, and masked email.

DETAILS

`getUserId` (routes/chat.ts:42-47) calls `security.decode`, which is `jws.decode(token)?.payload` (lib/insecurity.ts:56) and does not check the signature. A token with `alg: none` or any made-up signature and `data.id` set to someone else's id is accepted. `getOrderById` (lines 157-171) then loads that user's email, applies the same vowel-masking used when orders are stored (order.ts:168), and returns the whole order document when the masked values match. Two problems follow. (1) Identity is forged with no credentials. This is worse than F-023: no forgery trick is needed, because nothing verifies the token. (2) Vowel-masking is not unique. Emails that differ only in vowels (for example `bob@x.io` and `bab@x.ia`) give the same masked string, so a user can register a colliding address and read another person's orders. `getUserNameFromToken` (line 49) uses the same unverified decode, so a forged token also controls which name goes into the system prompt.

EVIDENCE

routes/chat.ts · lines 42–54

```
 42 export async function getUserId (req: Request): Promise<number | undefined> {
 43   const token = utils.jwtFrom(req)
 44   if (!token) return undefined
 45   const decoded = security.decode(token) as { data?: { id?: number } } |
undefined
 46   return decoded?.data?.id
 47 }
 48
 49 export async function getUserNameFromToken (req: Request): Promise<string |
undefined> {
 50   const userId = await getUserId(req)
 51   if (!userId) return undefined
 52   const user = await UserModel.findByPk(userId, { attributes: ['username'] })
 53   return user?.username ?? undefined
```

1 more line in the HTML report

routes/chat.ts · lines 157–171

```
157         execute: async ({ orderId }) => {
158           const userId = await getUserId(req)
159           if (!userId) return { error: 'Customer not authenticated' }
160
161           const user = await UserModel.findByPk(userId, { attributes: ['email']
})
162           if (!user) return { error: 'Customer not found' }
163
164           const maskedEmail = user.email ? user.email.replace(/[aeiou]/gi, '*')
: undefined
165           const order = await db.ordersCollection.findOne({ orderId })
166
167           if (!order) return { error: 'Order not found' }
168           if (order.email !== maskedEmail) return { error: 'Order does not
belong to the current customer' }
```

3 more lines in the HTML report

lib/insecurity.ts · line 56

```
 56 export const decode = (token: string) => { return jws.decode(token)?.payload }
```

routes/order.ts · lines 162–170

```
162           db.ordersCollection.insert({
163             promotionalAmount: discountAmount,
164             paymentId: req.body.orderDetails ? req.body.orderDetails.paymentId
: null,
165             addressId: req.body.orderDetails ? req.body.orderDetails.addressId
: null,
166             orderId,
167             delivered: false,
168             email: (email ? email.replace(/[aeiou]/gi, '*') : undefined),
169             totalPrice,
170             products: basketProducts,
```

REFERENCES

Relevant to: A04:2025 Cryptographic Failures; ASVS 5.0.0 V8.3 Operation Level Authorization.

Weakness: CWE-347.

Further reading: AI Agent Security Cheat Sheet.

CRITICAL F–055 **Unauthenticated chat endpoint has no rate limit, token cap or input cap**

LLM integrations · LLM-04 · effort M

RECOMMENDATION

Require authentication for /rest/chat (or a captcha for anonymous use). Add an express-rate-limit keyed on the verified user id with a daily token quota. Validate the body with zod: only `user`/`assistant` roles, at most around 20 messages, and a cap on total characters. Set `maxOutputTokens`, and lower `stepCountIs` to what the tools actually need. Alert on the existing token metrics.

Anyone on the internet can use the shop's chatbot as much as they like, with messages as long as they like, without an account. Every call costs money with a paid AI provider, so an attacker could run up a large bill or knock the chatbot offline.

LIKELIHOOD

Likely once the chatbot points at a paid API: a simple script can call it in a loop without signing in.

IMPACT

Unlimited LLM spend, or a self-hosted model kept fully busy, at the attacker's choice, which blocks real customers.

DETAILS

`POST /rest/chat` is mounted with no auth and no rate limiter (server.ts:632; the only rateLimit in server.ts covers reset-password). `streamText` is called with no `maxOutputTokens`

(routes/chat.ts:199-209). `req.body.messages` is used as-is (line 187), with no limit on the number of messages, their size or their roles, and the body parser accepts any `*/*` text. Each request can run up to 10 tool steps (`stepCountIs(10)`) plus `llmMaxRetries` retries, so one request can turn into many model calls. The token counters exist (lines 57-79), but nothing attributes usage to a user or enforces a budget.

EVIDENCE

server.ts · lines 631–632

```
631   /* Chat API endpoint */
632   app.post('/rest/chat', utils.asyncHandler(chat()))
```

routes/chat.ts · lines 186–209

```
186     const model = config.get<string>('application.chatBot.model')
187     const messages = req.body?.messages ?? []
188     const userName = await getUserNameFromToken(req)
189
190     res.setHeader('Content-Type', 'text/event-stream')
191     res.setHeader('Cache-Control', 'no-cache, no-transform')
192     res.setHeader('Connection', 'keep-alive')
193     res.setHeader('Content-Encoding', 'identity')
194     res.flushHeaders()
195
196     const systemPrompt = buildSystemPrompt(userName)
197
```

12 more lines in the HTML report

server.ts · lines 347–353

```
347   /* Rate limiting */
348   app.enable('trust proxy')
349   app.use('/rest/user/reset-password', rateLimit({
350     windowMs: 5 * 60 * 1000,
351     max: 100,
352     keyGenerator ({ headers, ip }: { headers: any, ip: any }) { return
headers['X-Forwarded-For'] ?? ip }
353   }))
```

REFERENCES

Weakness: CWE-770.

Further reading: Denial of Service Cheat Sheet.

HIGH F-012 **Detected a sequelize statement that is tainted by user-input.**

Security · SEC-04 · effort S

RECOMMENDATION

Confirm the input is attacker-controlled; if so, follow the rule's references.

Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it is recommended to use parameterized queries or prepared statements.

DETAILS

Semgrep rule rules.javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection.

EVIDENCE

routes/login.ts · line 31

```
31      models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })
```

REFERENCES

Relevant to: A05:2025 Injection; ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-89.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

HIGH F-015 **Detected a sequelize statement that is tainted by user-input.**

Security · SEC-04 · effort S

RECOMMENDATION

Confirm the input is attacker-controlled; if so, follow the rule's references.

Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it is recommended to use parameterized queries or prepared statements.

DETAILS

Semgrep rule rules.javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection.

EVIDENCE

routes/search.ts · line 22

```
22      models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)
```

REFERENCES

Relevant to: A05:2025 Injection; ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-89.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

HIGH F-017 **Five folders can be browsed by anyone, the encryption keys and server logs among them**

Security · SEC-15 · effort S

RECOMMENDATION

Confirm the input is attacker-controlled; if so, follow the rule's references.

Anyone can list and download the contents of /ftp, /encryptionkeys, /support/logs, /infrastructure and /.well-known. Among them are key files and the server's access logs.

DETAILS

Semgrep rule rules.javascript.express.security.audit.express-check-directory-listing.express-check-directory-listing.

EVIDENCE

server.ts · line 260

```
260   app.use('/infrastructure', serveIndexMiddleware, serveIndex('infrastructure', { icons: true, view: 'details', filter: (filename) => filename !== 'README.md' }))
```

server.ts · line 278

```
278   app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true }))
```

server.ts · line 282

```
282   app.use('/.well-known', serveIndexMiddleware, serveIndex('.well-known', { icons: true, view: 'details' }))
```

server.ts · line 286

```
286   app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', { icons: true, view: 'details' }))
```

server.ts · line 290

```
290   app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view: 'details' }))
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V15.3 Defensive Coding; ASVS 5.0.0 V1.5 Safe Deserialization.

Weakness: CWE-548.

Further reading: NodeJS Security Cheat Sheet; Deserialization Cheat Sheet; Prototype Pollution Prevention Cheat Sheet.

HIGH F-022 **Passwords stored as unsalted MD5 hashes**

Security · SEC-01 · effort M

RECOMMENDATION

Replace `hash` with bcrypt (cost ≥ 12) or argon2id. Verify with the library's constant-time compare in application code, not in SQL. Rehash on the user's next successful login and force a reset for accounts that never log in again.

Customer passwords are scrambled with a very old, fast method that offers almost no protection. If the database leaks, and several bugs in this app let it leak, most passwords can be recovered within minutes.

LIKELIHOOD

Anyone who gets a copy of the Users table (via the SQL injection in login/search, or the user list endpoints) gets these hashes.

IMPACT

Unsalted MD5 is cracked at billions of guesses per second or looked up in rainbow tables, so most user passwords are recovered and can be reused on other sites.

DETAILS

The User model's password setter calls `security.hash`, which is `crypto.createHash('md5')` with no salt or work factor. Login compares `security.hash(req.body.password)` in SQL, and change-password and 2FA setup compare against the same MD5 value. Identical passwords produce identical hashes across users.

EVIDENCE

lib/insecurity.ts · line 41

```
41  export const hash = (data: string) =>
crypto.createHash('md5').update(data).digest('hex')
```

models/user.ts · lines 67–72

```
67        password: {
68          type: DataTypes.STRING,
69          set (clearTextPassword: string) {
70            this.setDataValue('password', security.hash(clearTextPassword))
71          }
72        },
```

routes/login.ts · line 31

```
31      models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email
|| ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS
NULL`, { model: UserModel, plain: true })
```

REFERENCES

Relevant to: A04:2025 Cryptographic Failures; ASVS 5.0.0 V6.2 Password Security; ASVS 5.0.0 V6.3 General Authentication Security; ASVS 5.0.0 V6.4 Authentication Factor Lifecycle and Recovery; ASVS v5.0.0-11.4.2 (V11.4 Hashing and Hash-based Functions).

Weakness: CWE-916.

Further reading: Authentication Cheat Sheet; Password Storage Cheat Sheet; Forgot Password Cheat Sheet; NIST SP 800-63B-4 (Digital Identity Guidelines: Authentication).

HIGH F-024 **OAuth accounts use a password derived from their email address**

Security · SEC-01 · effort M

RECOMMENDATION

Do the OAuth code exchange and token verification on the server. Link the provider subject ID to the user record and issue the session there. Give OAuth-only accounts no usable password, or a random one the client never sees.

Accounts created with "Log in with Google" get a password anyone can work out from the email address. Knowing someone's email is enough to log in as them.

LIKELIHOOD

The derivation is in the shipped JavaScript bundle, so anyone who knows an OAuth user's email can compute the password.

IMPACT

Full takeover of every account created through Google sign-in, using the ordinary email/password login.

DETAILS

After the Google profile is fetched, the browser registers and logs in the user with `password = btoa(reversed email)`. The server has no notion of an OAuth identity. It accepts a normal `/rest/user/login` with that password, and the `oauth: true` flag in the body is ignored. Because the code runs on the client, the Google access token is never verified server-side.

EVIDENCE

frontend/src/app/oauth/oauth.component.ts · lines 28–52

```
28   ngOnInit (): void {
29
this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).subscribe({
30       next: (profile: any) => {
31         const password = btoa(profile.email.split('').reverse().join(''))
32         this.userService.save({ email: profile.email, password, passwordRepeat:
password }).subscribe({
33           next: () => {
34             this.login(profile)
35           },
36           error: () => { this.login(profile) }
37         })
38       },
39       error: (error) => {
```

13 more lines in the HTML report

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V6.2 Password Security; ASVS 5.0.0 V6.3 General Authentication Security; ASVS 5.0.0 V6.4 Authentication Factor Lifecycle and Recovery; ASVS v5.0.0-11.4.2 (V11.4 Hashing and Hash-based Functions).

Weakness: CWE-1391.

Further reading: Authentication Cheat Sheet; Password Storage Cheat Sheet; Forgot Password Cheat Sheet; NIST SP 800-63B-4 (Digital Identity Guidelines: Authentication).

HIGH F-025 **JWT carries password hash and TOTP secret; no revocation on logout**

Security · SEC-02 · effort M

RECOMMENDATION

Put only `sub`, `role` and minimal claims in the token. Deliver it in an HttpOnly, Secure, SameSite=Lax/Strict cookie set by the server, not localStorage. Add a server-side logout that invalidates the session (token ID deny-list or short-lived access token plus revocable refresh token).

The login token handed to the browser contains the user's scrambled password and their two-factor secret. It is stored where malicious scripts can read it, and logging out does not cancel it.

LIKELIHOOD

Every login issues such a token. It sits in localStorage and in a non-HttpOnly cookie, so any XSS in the app (several exist) can read it.

IMPACT

A stolen token reveals the user's MD5 password hash and 2FA seed and stays usable for 6 hours after logout.

DETAILS

`login()` signs `{ data: user, bid }`, where `user` is the full Users row from `SELECT *`, including `password` (MD5) and `totpSecret`. 2FA `verify` does the same with `plainUser`. JWT payloads are only base64, not encrypted. The frontend writes the token to `localStorage` and to a cookie via ngx-cookie without HttpOnly, Secure or SameSite, and the server also sets `res.cookie('token', token)` without flags in `updateAuthenticatedUsers`. Logout only deletes the client copies. `authenticatedUsers.tokenMap` is never purged and there is no deny-list, so the token stays valid until `exp`.

EVIDENCE

routes/login.ts · lines 18–24

```
18   function afterLogin (user: User, res: Response, next: NextFunction) {
19     BasketModel.findOrCreate({ where: { UserId: user.id } })
20       .then(([basket]: [BasketModel, boolean]) => {
21         const authenticatedUser = { data: user, bid: basket.id } // keep track
of original basket
22         const token = security.authorize(authenticatedUser)
23         security.authenticatedUsers.put(token, authenticatedUser)
24         res.json({ authentication: { token, bid: basket.id, umail: user.email }
})
```

routes/2fa.ts · lines 41–44

```
41      const token = security.authorize(plainUser)
42      // @ts-expect-error FIXME set new property for original basket
43      plainUser.bid = basket.id // keep track of original basket for challenge
solution check
44      security.authenticatedUsers.put(token, plainUser)
```

lib/insecurity.ts · lines 184–195

```
184 export const updateAuthenticatedUsers = () => (req: Request, res: Response,
next: NextFunction) => {
185   const token = req.cookies.token || utils.jwtFrom(req)
186   if (token && authenticatedUsers.get(token) === undefined) {
187     jwt.verify(token, publicKey, (err: Error | null, decoded: any) => {
188       if (err === null && decoded?.data !== undefined) {
189         authenticatedUsers.put(token, decoded)
190         res.cookie('token', token)
191       }
192     })
193   }
194   next()
195 }
```

frontend/src/app/login/login.component.ts · lines 100–106

```
100       next: (authentication: any) => {
101         const redirectUrl =
this.route.snapshot.queryParamMap.get('redirectUrl') ?? '/search'
102         localStorage.setItem('token', authentication.token)
103         const expires = new Date()
104         expires.setHours(expires.getHours() + 8)
105         this.cookieService.put('token', authentication.token, { expires })
106         sessionStorage.setItem('bid', authentication.bid)
```

frontend/src/app/navbar/navbar.component.ts · lines 241–250

```
241   logout () {
242     this.userService.saveLastLoginIp().subscribe({ next: () => { this.noop() },
error: (err) => { console.log(err) } })
243     localStorage.removeItem('token')
244     this.cookieService.remove('token')
245     sessionStorage.removeItem('bid')
246     sessionStorage.removeItem('itemTotal')
247     sessionStorage.removeItem('guestBasket')
248     this.userService.isLoggedIn.next(false)
249     this.ngZone.run(async () => await this.router.navigate(['/']))
250   }
```

REFERENCES

Relevant to: A06:2025 Insecure Design; ASVS 5.0.0 V7.2 Fundamental Session Management Security; ASVS 5.0.0 V7.3 Session Timeout; ASVS 5.0.0 V7.4 Session Termination; ASVS 5.0.0 V9.1 Token source and integrity; ASVS 5.0.0 V3.3 Cookie Setup.

Weakness: CWE-522.

Further reading: Session Management Cheat Sheet.

HIGH F-026 **Password change skips current-password check and sends passwords in URL**

Security · SEC-01 · effort S

RECOMMENDATION

Make it a POST with a JSON body. Always require and verify the current password (or a recent re-authentication). Invalidate other sessions after a change. Keep secrets out of URLs, and scrub query strings from access logs.

Changing a password does not really require the old one, so someone who briefly hijacks a session can lock the owner out for good. New passwords are also written in plain text into server log files.

LIKELIHOOD

Anyone holding a user's token, for example via the XSS issues in this app, can call it. The password also lands in access logs on every legitimate change.

IMPACT

Permanent account takeover from a short-lived token theft, and plaintext new passwords stored in logs/access.log, which is publicly browsable under /support/logs.

DETAILS

`GET /rest/user/change-password` reads `current`, `new` and `repeat` from the query string. The current password is checked only `if (currentPassword && ...)`, so omitting `current` skips the check. Because it is a GET, morgan's `combined` format writes the full URL, including both passwords, to `logs/access.log.*`. That directory is served by `/support/logs` (see F-020/F-011).

EVIDENCE

routes/changePassword.ts · lines 13–42

```
13   return async ({ query, headers, connection }: Request, res: Response, next:
NextFunction) => {
14     const currentPassword = query.current as string
15     const newPassword = query.new as string
16     const newPasswordInString = newPassword?.toString()
17     const repeatPassword = query.repeat
18
19     if (!newPassword || newPassword === 'undefined') {
20       res.status(401).send(res.__('Password cannot be empty.'))
21       return
22     } else if (newPassword !== repeatPassword) {
23       res.status(401).send(res.__('New and repeated password do not match.'))
24       return
```

18 more lines in the HTML report

server.ts · line 591

```
591   app.get('/rest/user/change-password', utils.asyncHandler(changePassword()))
```

server.ts · line 345

```
345   app.use(morgan('combined', { stream: accessLogStream }))
```

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V6.2 Password Security; ASVS 5.0.0 V6.3 General Authentication Security; ASVS 5.0.0 V6.4 Authentication Factor Lifecycle and Recovery; ASVS v5.0.0-11.4.2 (V11.4 Hashing and Hash-based Functions).

Weakness: CWE-620.

Further reading: Authentication Cheat Sheet; Password Storage Cheat Sheet; Forgot Password Cheat Sheet; NIST SP 800-63B-4 (Digital Identity Guidelines: Authentication).

HIGH F-027 **Password reset relies on guessable security question with spoofable throttle**

Security · SEC-01 · effort M

RECOMMENDATION

Replace security questions with a single-use, time-limited reset link emailed to the address on file. Rate-limit login, reset, security-question and 2FA verify per account and per real client IP: configure `trust proxy` to the exact number of proxy hops and don't key on a raw header. Return identical responses for unknown emails.

To reset someone's password you only need their email and the answer to a personal question like "your pet's name". No email is sent and the attempt limit can be dodged, so accounts can be taken over by guessing.

LIKELIHOOD

Security answers (mother's maiden name, first pet...) are often public or guessable, and the 100-per-5-minutes limit is bypassed by changing the X-Forwarded-For header.

IMPACT

Takeover of any account whose email is known, with no email ownership proof.

DETAILS

`resetPassword` changes the password when `hmac(answer)` matches the stored security answer. No token is emailed to the account owner. `/rest/user/security-question?email=` returns the question for any registered email and `{}` otherwise, which enumerates accounts and tells an attacker what to guess. The only throttle is `rateLimit` with `keyGenerator` returning `headers['X-Forwarded-For'] ?? ip` while `trust proxy` is enabled. A client can send arbitrary X-Forwarded-For values, so each attempt gets a fresh bucket. `/rest/user/login` has no rate limit or lockout at all, and the 2FA verify limiter (100 per 5 minutes per IP) allows brute forcing six-digit TOTP codes over time.

EVIDENCE

routes/resetPassword.ts · lines 34–49

```
34      try {
35        const data = await SecurityAnswerModel.findOne({
36          include: [{
37            model: UserModel,
38            where: { email }
39          }]
40        })
41        if ((data != null) && security.hmac(answer) === data.answer) {
42          const user = await UserModel.findByPk(data.UserId)
43          if (user) {
44            const updatedUser = await user.update({ password: newPassword })
45            res.json({ user: updatedUser })
```

4 more lines in the HTML report

routes/securityQuestion.ts · lines 12–26

```
12   return async ({ query }: Request, res: Response, next: NextFunction) => {
13     const email = query.email
14     try {
15       const answer = await SecurityAnswerModel.findOne({
16         include: [{
17           model: UserModel,
18           where: { email: email?.toString() }
19         }]
20       })
21       if (answer != null) {
22         const question = await
SecurityQuestionModel.findByPk(answer.SecurityQuestionId)
23         res.json({ question })
```

3 more lines in the HTML report

server.ts · lines 348–353

```
348   app.enable('trust proxy')
349   app.use('/rest/user/reset-password', rateLimit({
350     windowMs: 5 * 60 * 1000,
351     max: 100,
352     keyGenerator ({ headers, ip }: { headers: any, ip: any }) { return
headers['X-Forwarded-For'] ?? ip }
353   }))
```

server.ts · lines 455–458

```
455   app.post('/rest/2fa/verify',
456     rateLimit({ windowMs: 5 * 60 * 1000, max: 100, validate: false }),
457     utils.asyncHandler(twoFactorAuth.verify)
458   )
```

server.ts · line 590

```
590   app.post('/rest/user/login', login())
```

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V6.2 Password Security; ASVS 5.0.0 V6.3 General Authentication Security; ASVS 5.0.0 V6.4 Authentication Factor Lifecycle and Recovery; ASVS v5.0.0-11.4.2 (V11.4 Hashing and Hash-based Functions).

Weakness: CWE-640.

Further reading: Authentication Cheat Sheet; Password Storage Cheat Sheet; Forgot Password Cheat Sheet; NIST SP 800-63B-4 (Digital Identity Guidelines: Authentication).

HIGH F-028 **Any user can read, modify and check out other users' baskets**

Security · SEC-03 · effort M

RECOMMENDATION

Derive the basket from the authenticated user (`BasketModel.findOne({ where: { id, UserId: user.id } })`) in every basket, checkout and coupon handler. Add finale `before` hooks (or custom routes) that scope BasketItem reads and writes to the caller's basket. Use the standard JSON body parser and reject duplicate keys.

Shopping baskets are not tied to their owners on the server. A logged-in customer can change a number in the request to see or alter someone else's basket and even place orders from it.

LIKELIHOOD

Basket IDs are small sequential integers, so any logged-in user can iterate them.

IMPACT

Other customers' basket contents are exposed, items can be added or removed and coupons applied in their baskets, and their baskets can be checked out with the attacker's wallet or no payment at all.

DETAILS

`/rest/basket/:id` only requires a valid JWT. `retrieveBasket`, `placeOrder` and `applyCoupon` look the basket up by `req.params.id` and never compare it to the caller's `bid` or `UserId`. The generated finale endpoints `/api/BasketItems/:id` (GET/PUT/DELETE) only require authentication, with no ownership check. `addBasketItem` checks `basketIds[0]` against `user.bid` but saves `basketIds[basketIds.length - 1]`, so sending `BasketId` twice (HTTP parameter pollution through the custom JSON parser) adds items to any basket.

EVIDENCE

routes/basket.ts · lines 16–27

```
16    return async (req: Request, res: Response, next: NextFunction) => {
17      try {
18        const id = req.params.id
19        const basket = await BasketModel.findOne({ where: { id }, include: [{
model: ProductModel, paranoid: false, as: 'Products' }] })
20        /* jshint eqeqeq:false */
21        if (((basket?.Products) != null) && basket.Products.length > 0) {
22          for (let i = 0; i < basket.Products.length; i++) {
23            basket.Products[i].name = req.__(basket.Products[i].name)
24          }
25        }
26
27        res.json(utils.queryResultToJson(basket))
```

routes/order.ts · lines 33–39

```
33    return (req: Request, res: Response, next: NextFunction) => {
34      const id = req.params.id
35      BasketModel.findOne({ where: { id }, include: [{ model: ProductModel,
paranoid: false, as: 'Products' }] })
36        .then(async (basket: BasketModel | null) => {
37          if (basket != null) {
38            const customer = security.authenticatedUsers.from(req)
39            const email = customer ? customer.data ? customer.data.email : '' :
''
```

routes/coupon.ts · lines 11–24

```
11    return async ({ params }: Request, res: Response, next: NextFunction) => {
12      try {
13        const id = params.id
14        let coupon: string | undefined | null = params.coupon ?
decodeURIComponent(params.coupon) : undefined
15        const discount = security.discountFromCoupon(coupon)
16        coupon = discount ? coupon : null
17
18        const basket = await BasketModel.findByPk(id)
19        if (!basket) {
20          next(new Error(`Basket with id=${id} does not exist.`))
21          return
22        }
```

2 more lines in the HTML report

routes/basketItems.ts · lines 21–46

```
21    const result = utils.parseJsonCustom((req as RequestWithRawBody).rawBody)
22    const productIds = []
23    const basketIds = []
24    const quantities = []
25
26    for (let i = 0; i < result.length; i++) {
27      if (result[i].key === 'ProductId') {
28        productIds.push(result[i].value)
29      } else if (result[i].key === 'BasketId') {
30        basketIds.push(result[i].value)
31      } else if (result[i].key === 'quantity') {
32        quantities.push(result[i].value)
```

14 more lines in the HTML report

server.ts · lines 359–362

```
359  app.use('/rest/basket', security.isAuthorized(), security.appendUserId())
360  /* BasketItems: API only accessible for authenticated users */
361  app.use('/api/BasketItems', security.isAuthorized())
362  app.use('/api/BasketItems/:id', security.isAuthorized())
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V8.2 General Authorization Design; ASVS 5.0.0 V8.3 Operation Level Authorization; ASVS 5.0.0 V14.2 General Data Protection.

Weakness: CWE-639.

Further reading: Authorization Cheat Sheet; Insecure Direct Object Reference Prevention Cheat Sheet.

HIGH F-029 **Unvalidated amounts let users mint wallet credit and negative orders**

Security · SEC-09 · effort M

RECOMMENDATION

Validate request bodies with a schema (zod/joi): positive integer quantities (also as a model `validate: { min: 1 }`), and positive, bounded top-up amounts. Credit the wallet only after a confirmed payment-provider charge. Reject orders whose computed total is ≤ 0.

The wallet top-up adds whatever amount the browser asks for without charging anything, and baskets accept negative quantities. Customers can give themselves unlimited store credit and free goods.

LIKELIHOOD

Any registered user with a saved card (any number is accepted) can send the request directly.

IMPACT

Free store credit and goods: wallet top-ups are never charged, and negative basket quantities produce negative order totals that credit the wallet.

DETAILS

`addWalletBalance` checks only that `paymentId` is one of the caller's cards, then runs `WalletModel.increment({ balance: req.body.balance })` with an unvalidated number. No payment is taken, and negative or huge values are accepted. `BasketItem.quantity` is a bare INTEGER with no `min` validator, and `quantityCheck` only compares upper limits. In `placeOrder`, `itemTotal = itemPrice * quantity` can be negative. With `paymentId === 'wallet'` the check `wallet.balance >= totalPrice` passes and `decrement` by a negative total increases the balance. `deliveryMethodId` and the coupon campaign data also come straight from the body.

EVIDENCE

routes/wallet.ts · lines 21–35

```
21 export function addWalletBalance () {
22   return async (req: Request, res: Response, next: NextFunction) => {
23     const cardId = req.body.paymentId
24     const card = cardId ? await CardModel.findOne({ where: { id: cardId,
UserId: req.body.UserId } }) : null
25     if (card != null) {
26       try {
27         await WalletModel.increment({ balance: req.body.balance }, { where: {
UserId: req.body.UserId } })
28         res.status(200).json({ status: 'success', data: req.body.balance })
29       } catch {
30         res.status(404).json({ status: 'error' })
31       }
32     } else {
```

3 more lines in the HTML report

models/basketitem.ts · line 39

```
39       quantity: DataTypes.INTEGER
```

routes/order.ts · line 92

```
92               const itemTotal = itemPrice * BasketItem.quantity
```

routes/order.ts · lines 144–155

```
144           if (req.body.UserId) {
145             if (req.body.orderDetails && req.body.orderDetails.paymentId ===
'wallet') {
146               const wallet = await WalletModel.findOne({ where: { UserId:
req.body.UserId } })
147               if ((wallet != null) && wallet.balance >= totalPrice) {
148                 await WalletModel.decrement({ balance: totalPrice }, { where: {
UserId: req.body.UserId } })
149               } else {
150                 next(new Error('Insufficient wallet balance.'))
151                 return
152               }
153             }
154             try {
155               await WalletModel.increment({ balance: totalPoints }, { where: {
UserId: req.body.UserId } })
```

REFERENCES

Relevant to: A05:2025 Injection; ASVS 5.0.0 V2.2 Input Validation; ASVS v5.0.0-15.3.3 (V15.3 Defensive Coding).

Weakness: CWE-20.

Further reading: Input Validation Cheat Sheet; Mass Assignment Cheat Sheet.

HIGH F-032 **B2B order lines evaluated as code in node:vm with notevil**

Security · SEC-04 · effort S

RECOMMENDATION

Parse order lines as JSON and validate them with a schema. Remove `vm`/`notevil` completely.

The business-order endpoint treats part of the order as a program and runs it. Attackers can use this to freeze the shop or potentially take over the server.

LIKELIHOOD

Any authenticated user (accounts are free; JWTs are forgeable per F-023) can post `orderLinesData`.

IMPACT

An infinite loop blocks the event loop for 2 s per request, which is easy denial of service, and `node:vm` plus the unmaintained `notevil` is a known escape surface leading to code execution.

DETAILS

`b2bOrder` places `body.orderLinesData` into a `vm` context and runs `safeEval(orderLinesData)`. Node's documentation states `vm` is not a security mechanism. `notevil` 1.3.x is abandoned and has published sandbox escapes. The 2-second timeout is synchronous, so each request stalls every other user.

EVIDENCE

routes/b2bOrder.ts · lines 17–32

```
17    return ({ body }: Request, res: Response, next: NextFunction) => {
18      if (utils.isChallengeEnabled(challenges.ch143cb02f) ||
utils.isChallengeEnabled(challenges.ch0ca8dfb9)) {
19        const orderLinesData = body.orderLinesData || ''
20        try {
21          const sandbox = { safeEval, orderLinesData }
22          vm.createContext(sandbox)
23          vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })
24          res.json({ cid: body.cid, orderNo: uniqueOrderNumber(), paymentDue:
dateTwoWeeksFromNow() })
25        } catch (err) {
26          if (utils.getErrorMessage(err).match(/Script execution timed out.*/) !=
null) {
27            res.status(503)
28            next(new Error('Sorry, we are temporarily not available! Please try
again later.'))
```

4 more lines in the HTML report

REFERENCES

Relevant to: A05:2025 Injection; ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-94.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

HIGH F-033 **Order tracking builds a MarsDB $where JavaScript expression from the URL**

Security · SEC-04 · effort S

RECOMMENDATION

Use an equality filter `find({ orderId: String(req.params.id) })` and drop `$where`. Return orders only to their owner.

The order tracking page passes the order number straight into a piece of code the database runs. Anyone, without logging in, can list every customer's orders or stall the server.

LIKELIHOOD

Unauthenticated: anyone can call /rest/track-order/:id. The truncation still leaves 60 characters of injected JS.

IMPACT

The injected expression runs server-side for every order, enough to dump all orders (`' || true || '`) or hang the process with a loop.

DETAILS

`trackOrder` runs `ordersCollection.find({ $where: \`this.orderId === '${id}'\` })`. When the related challenge flag is enabled, `id` is only truncated to 60 characters, not sanitised. `$where` is evaluated as JavaScript by MarsDB. A payload such as `x' || true || '` returns every order (emails are only vowel-masked, plus products and totals).

EVIDENCE

routes/trackOrder.ts · lines 13–22

```
13    return (req: Request, res: Response) => {
14      // Truncate id to avoid unintentional RCE
15      const id = !utils.isChallengeEnabled(challenges.ch46bea641) ?
String(req.params.id).replace(/[^\w-]+/g, '') : utils.trunc(req.params.id, 60)
16
17      db.ordersCollection.find({ $where: `this.orderId === '${id}'`
}).then((order: any) => {
18        const result = utils.queryResultToJson(order)
19        if (result.data[0] === undefined) {
20          result.data[0] = { orderId: id }
21        }
22        res.json(result)
```

REFERENCES

Relevant to: ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-943.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

HIGH F-034 **Review endpoints: $where injection, operator injection and unowned edits**

Security · SEC-04 · effort S

RECOMMENDATION

Use `find({ product: Number(id) })`. Coerce `_id` to a string and reject objects. Filter updates with `{ _id, author: user.data.email }` and drop `multi`. Require auth on create and take the author from the session. Make likes atomic (`$addToSet` plus conditional `$inc`). Remove the global `sleep`.

Product reviews are poorly protected. Anyone can freeze the server through the review page, any logged-in user can rewrite every review in the shop at once, and reviews can be posted pretending to be another customer.

LIKELIHOOD

Show is unauthenticated; edit needs any account. Payloads such as `{"id":{"$ne":-1}}` are trivial.

IMPACT

One request overwrites every product review in the shop. The read endpoint also allows a blocking `sleep()` DoS, and reviews can be posted under anyone's name.

DETAILS

`showProductReviews` builds `$where: 'this.product == ' + id`. With the challenge flag on, `id` is the URL segment truncated to 40 characters, and the code installs a global blocking `sleep()` reachable from it. `updateProductReviews` passes `req.body.id` straight into the `_id` filter with `multi: true`, so an object like `{"$ne": -1}` matches every review. It also never checks that the caller authored the review (`user` is fetched and ignored). `createProductReviews` (PUT, no auth middleware) takes `author` from the body instead of the session. `likeProductReviews` has a read-modify-write race (the 150 ms sleep makes it wide), so a user can like the same review many times.

EVIDENCE

routes/showProductReviews.ts · lines 17–36

```
17 global.sleep = (time: number) => {
18   // Ensure that users don't accidentally dos their servers for too long
19   if (time > 2000) {
20     time = 2000
21   }
22   const stop = new Date().getTime()
23   while (new Date().getTime() < stop + time) {
24     ;
25   }
26 }
27
28 export function showProductReviews () {
```

8 more lines in the HTML report

routes/updateProductReviews.ts · lines 15–20

```
15     const user = security.authenticatedUsers.from(req)
16     db.reviewsCollection.update(
17       { _id: req.body.id },
18       { $set: { message: req.body.message } },
19       { multi: true }
20     ).then(
```

routes/createProductReviews.ts · lines 16–25

```
16     const user = security.authenticatedUsers.from(req)
17
18     try {
19       await reviewsCollection.insert({
20         product: req.params.id,
21         message: req.body.message,
22         author: req.body.author,
23         likesCount: 0,
24         likedBy: []
25       })
```

routes/likeProductReviews.ts · lines 30–52

```
30        const likedBy = review.likedBy
31        if (likedBy.includes(user.data.email)) {
32          return res.status(403).json({ error: 'Not allowed' })
33        }
34
35        await db.reviewsCollection.update(
36          { _id: id },
37          { $inc: { likesCount: 1 } }
38        )
39
40        // Artificial wait for timing attack challenge
41        await sleep(150)
```

11 more lines in the HTML report

REFERENCES

Relevant to: ASVS 5.0.0 V1.2 Injection Prevention.

Weakness: CWE-943.

Further reading: Injection Prevention Cheat Sheet; SQL Injection Prevention Cheat Sheet; OS Command Injection Defense Cheat Sheet; NodeJS Security Cheat Sheet.

HIGH F-037 **Profile image URL is fetched server-side without any allowlist**

Security · SEC-07 · effort M

RECOMMENDATION

Accept only https URLs. Resolve DNS and reject private, loopback, link-local and metadata ranges, and re-check on each redirect (or set `redirect: 'manual'`). Verify the content type is an image and cap the size. Preferably fetch through an egress proxy with an allowlist.

The "profile picture from a link" feature makes the server download whatever address a user gives it, including internal systems that should never be reachable from outside. The result can then be read back as an image.

LIKELIHOOD

Any logged-in user can submit any URL; redirects are followed by default.

IMPACT

The server makes requests to internal services and cloud metadata endpoints (e.g. 169.254.169.254) on the attacker's behalf, and the response body is saved as a publicly served image.

DETAILS

`profileImageUrlUpload` calls `fetch(url)` with `req.body.imageUrl` unchecked (scheme, host, IP range), follows redirects, and writes the body to `frontend/dist/.../uploads/<id>.<ext>`, where it is publicly readable. Full-read SSRF. On failure the raw URL is stored as `profileImage`, which is later interpolated into the profile page's Content-Security-Policy (see the CSP finding).

EVIDENCE

routes/profileImageUrlUpload.ts · lines 18–37

```
18      if (req.body.imageUrl !== undefined) {
19        const url = req.body.imageUrl
20        if (url.match(/(.)*solve\/challenges\/server-side(.)*/) !== null)
req.app.locals.abused_ssrf_bug = true
21        const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
22        if (loggedInUser) {
23          try {
24            const response = await fetch(url)
25            if (!response.ok || !response.body) {
26              throw new Error('url returned a non-OK status code or an empty
body')
27            }
28            const ext = ['jpg', 'jpeg', 'png', 'svg',
'gif'].includes(url.split('.').slice(-1)[0].toLowerCase()) ?
url.split('.').slice(-1)[0].toLowerCase() : 'jpg'
29            const fileStream =
fs.createWriteStream(`frontend/dist/frontend/assets/public/images/uploads/${loggedIn
User.data.id}.${ext}`, { flags: 'w' })
```

8 more lines in the HTML report

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS v5.0.0-1.3.6 (V1.3 Sanitization).

Weakness: CWE-918.

Further reading: Server-Side Request Forgery Prevention Cheat Sheet.

HIGH F-042 **Angular sanitizer bypassed on search term, feedback, emails, IPs and orders**

Security · SEC-05 · effort M

RECOMMENDATION

Remove every `bypassSecurityTrustHtml` on data. Use text interpolation (`{{ }}`) and CSS classes instead of building HTML strings. Where rich text is needed, sanitise with a current DOMPurify on output. Upgrade sanitize-html. Ignore `True-Client-IP` unless set by a trusted proxy. Add a strict CSP (see SEC-11).

Several pages deliberately switch off the browser-side protection against malicious scripts. A crafted link, or a crafted review, email address or IP header, can run attacker code in other users' and admins' browsers and steal their logins.

LIKELIHOOD

The search sink is a reflected DOM XSS via a link (`/#/search?q=<iframe src="javascript:..."></iframe>`). The others are stored XSS fed by unauthenticated or self-service inputs.

IMPACT

Script runs in victims' sessions, including admins. Tokens sit in localStorage and non-HttpOnly cookies, so this means account takeover.

DETAILS

`bypassSecurityTrustHtml` is applied to: the `q` query parameter (search-result 140, reflected); product descriptions (search-result 110), which are writable unauthenticated via PUT /api/Products (F-039); feedback comments on the About carousel and the admin page; user emails on the admin page (built into an HTML string with interpolation); `lastLoginIp` taken from the attacker-controlled `True-Client-IP` header (saveLoginIp stores it unsanitised when the challenge flag is on); and track-order IDs. Product details render `description` via `[innerHTML]`. Server-side, models rely on `sanitize-html` 1.4.2 (2014, with known bypasses), and the feedback model uses the single-pass `sanitizeHtml` variant, which nested payloads defeat.

EVIDENCE

frontend/src/app/search-result/search-result.component.ts · lines 108–140

```
108   trustProductDescription (tableData: any[]) {
109     for (let i = 0; i < tableData.length; i++) {
110       tableData[i].description =
this.sanitizer.bypassSecurityTrustHtml(tableData[i].description)
111     }
112   }
113
114
115   ngOnDestroy () {
116     if (this.routerSubscription) {
117       this.routerSubscription.unsubscribe()
118     }
119
```

18 more lines in the HTML report

frontend/src/app/administration/administration.component.ts · lines 73–92

```
 73         for (const user of this.userDataSource) {
 74           user.email = this.sanitizer.bypassSecurityTrustHtml(`<span
class="${this.doesUserHaveAnActiveSession(user) ? 'confirmation' :
'error'}">${user.email}</span>`)
 75         }
 76         this.userDataSource = new MatTableDataSource(this.userDataSource)
 77         this.userDataSource.paginator = this.paginatorUsers
 78         this.resultsLengthUser = users.length
 79       },
 80       error: (err) => {
 81         this.error = err
 82         console.log(this.error)
 83       }
 84     })
```

8 more lines in the HTML report

frontend/src/app/about/about.component.ts · lines 117–122

```
117           feedbacks[i].comment = `<figcaption><p class="feedback-comment">${
118             feedbacks[i].comment
119           }</p><div class="feedback-stars">(${this.stars[feedbacks[i].rating]})
</div></figcaption>`
120           feedbacks[i].comment = this.sanitizer.bypassSecurityTrustHtml(
121             feedbacks[i].comment
122           )
```

frontend/src/app/last-login-ip/last-login-ip.component.ts · lines 38–40

```
 38       if (payload.data.lastLoginIp) {
 39
 40         this.lastLoginIp =
this.sanitizer.bypassSecurityTrustHtml(`<small>${payload.data.lastLoginIp}</small>`)
```

routes/saveLoginIp.ts · lines 18–31

```
18        let lastLoginIp = req.headers['true-client-ip']
19        if (Array.isArray(lastLoginIp)) {
20          lastLoginIp = lastLoginIp[0]
21        }
22        if (utils.isChallengeEnabled(challenges.ch29ff72e7)) {
23        } else {
24          lastLoginIp = security.sanitizeSecure(lastLoginIp ?? '')
25        }
26        if (lastLoginIp === undefined) {
27          lastLoginIp = utils.toSimpleIpAddress(req.socket.remoteAddress ?? '')
28        }
29        try {
```

2 more lines in the HTML report

REFERENCES

Relevant to: A05:2025 Injection; ASVS 5.0.0 V1.1 Encoding and Sanitization Architecture; ASVS 5.0.0 V1.3 Sanitization; ASVS 5.0.0 V3.2 Unintended Content Interpretation.

Weakness: CWE-79.

Further reading: Cross Site Scripting Prevention Cheat Sheet; DOM based XSS Prevention Cheat Sheet.

HIGH F-043 **Data-erasure form spreads request body into render options, allowing file read**

Security · SEC-08 · effort S

RECOMMENDATION

Never spread `req.body` into template locals or options. Pass only the explicit fields needed (`email`, `securityAnswer`) and fix the layout server-side. Add CSRF protection to this cookie-authenticated POST.

The "delete my data" page lets a logged-in user name any file on the server and see the beginning of it. That can expose configuration and stored secrets.

LIKELIHOOD

Any logged-in user can POST `layout=../../../etc/passwd` (or any path) to /dataerasure.

IMPACT

The first 100 characters of arbitrary server files are disclosed. The deny-list covers only three substrings, so config, the SQLite DB, logs and .env files remain readable.

DETAILS

`res.render('dataErasureResult', { ...req.body, ...themeVars })` passes user input as hbs render options. hbs honours the `layout` option as a file path, and the only check is a substring deny-list of `ftp`, `ctf.key` and `encryptionkeys` after `path.resolve`. The handler is cookie-authenticated with no CSRF token, so a cross-site form can also file deletion requests for a victim.

EVIDENCE

routes/dataErasure.ts · lines 103–117

```
103       if (req.body.layout && utils.isChallengeEnabled(challenges.ch4086891b)) {
104         const filePath: string = path.resolve(req.body.layout).toLowerCase()
105         const isForbiddenFile: boolean = (filePath.includes('ftp') ||
filePath.includes('ctf.key') || filePath.includes('encryptionkeys'))
106         if (!isForbiddenFile) {
107           res.render('dataErasureResult', {
108             ...req.body,
109             ...themeVars
110           }, (error, html) => {
111             if (!html || error) {
112               next(new Error(error.message))
113             } else {
114               const sendlfrResponse: string = html.slice(0, 100) + '......'
```

3 more lines in the HTML report

REFERENCES

Relevant to: A06:2025 Insecure Design; ASVS 5.0.0 V5.2 File Upload and Content; ASVS 5.0.0 V5.3 File Storage; ASVS 5.0.0 V5.4 File Download.

Weakness: CWE-73.

Further reading: File Upload Cheat Sheet.

HIGH F-046 **Discount coupons are unsigned encodings anyone can forge**

Security · SEC-14 · effort M

RECOMMENDATION

Store issued coupons server-side (code, discount, expiry, redemption count) and look them up, or sign them with an HMAC key from a secret store and verify with `timingSafeEqual`. Move the security-answer HMAC key to an environment secret, or better, retire security questions (see F-027).

Discount codes are just a reversible scrambling of the month and the percentage, with no secret involved. Anyone who works this out can make their own 99%-off code.

LIKELIHOOD

The format (z85 of "MMMYY-NN") is visible from any one real coupon and from the client code; no account knowledge is needed.

IMPACT

Customers can apply any discount up to 99% on any order during the current month, which is direct revenue loss.

DETAILS

`generateCoupon` returns `z85.encode(toMMMYY(date) + '-' + discount)`, and `discountFromCoupon` decodes it and accepts any two-digit discount whose month matches the current one. There is no MAC, server-side coupon table or single-use tracking. Relatedly, security answers are HMACed with a hard-coded key in source (`'pa4qacea4VK9t9nGv7yZtwmj'`), so a DB leak plus the repo allows offline guessing, and order IDs embed `md5(email).slice(0,4)`.

EVIDENCE

lib/insecurity.ts · lines 97–119

```
 97 export const generateCoupon = (discount: number, date = new Date()) => {
 98   const coupon = utils.toMMMYY(date) + '-' + discount
 99   return z85.encode(coupon)
100 }
101
102 export const discountFromCoupon = (coupon?: string) => {
103   if (!coupon) {
104     return undefined
105   }
106   const decoded = z85.decode(coupon)
107   if (decoded && (hasValidFormat(decoded.toString()) != null)) {
108     const parts = decoded.toString().split('-')
```

11 more lines in the HTML report

lib/insecurity.ts · line 42

```
 42 export const hmac = (data: string) => crypto.createHmac('sha256',
'pa4qacea4VK9t9nGv7yZtwmj').update(data).digest('hex')
```

REFERENCES

Relevant to: A08:2025 Software or Data Integrity Failures; ASVS 5.0.0 V11.3 Encryption Algorithms; ASVS 5.0.0 V11.4 Hashing and Hash-based Functions; ASVS 5.0.0 V11.5 Random Values.

Weakness: CWE-345.

Further reading: Cryptographic Storage Cheat Sheet.

HIGH F-048 **Customer order PDFs written to the public /ftp folder; extension check bypassable**

Security · SEC-08 · effort S

RECOMMENDATION

Store order PDFs outside any served directory and stream them only to the owning user after an auth check. Remove the /ftp listing and static serving. Strip or reject `%00` before validation and validate the final resolved path. Delete backup and credential files from the repository.

Each order confirmation, with the customer's email and what they bought, is saved into a public download folder that anyone can browse. Confidential backup files in that folder can be downloaded with a simple trick.

LIKELIHOOD

/ftp is browsable without login, so anyone can list and download every order confirmation.

IMPACT

Every customer's email, purchased items, prices and order ID are exposed. Backup files (package.json.bak, coupons_2013.md.bak, the KeePass database) are downloadable via `%2500`.

DETAILS

This goes beyond scanner leads F-018 (directory listing on /ftp) and F-009 (sendFile path traversal). `placeOrder` writes `order_<id>.pdf` containing the customer email and line items into `ftp/`, which `serveIndex('ftp')` lists and `servePublicFiles` serves. In `servePublicFiles`, the `.md`/`.pdf` allow-list runs before `cutOffPoisonNullByte`, so `/ftp/package.json.bak%2500.md` passes the check and then serves `package.json.bak`. The folder also ships `incident-support.kdbx`, explicitly allow-listed.

EVIDENCE

routes/order.ts · lines 40–45

```
 40           const orderId = security.hash(email).slice(0, 4) + '-' +
utils.randomHexString(16)
 41           const pdfFile = `order_${orderId}.pdf`
 42           const { default: PDFDocument } = await import('pdfkit')
 43           const doc = new PDFDocument()
 44           const date = new Date().toJSON().slice(0, 10)
 45           const fileWriter = doc.pipe(fs.createWriteStream(path.join('ftp/',
pdfFile)))
```

routes/fileServer.ts · lines 25–44

```
25   function verify (file: string, res: Response, next: NextFunction) {
26     if (file && (endsWithAllowlistedFileType(file) || (file === 'incident-
support.kdbx'))) {
27       file = security.cutOffPoisonNullByte(file)
28
29       verifySuccessfulPoisonNullByteExploit(file)
30
31       res.sendFile(path.resolve('ftp/', file))
32     } else {
33       res.status(403)
34       next(new Error('Only .md and .pdf files are allowed!'))
35     }
36   }
```

8 more lines in the HTML report

server.ts · lines 278–279

```
278   app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true }))
279   app.use('/ftp(?!/quarantine)/:file', servePublicFiles())
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V5.2 File Upload and Content; ASVS 5.0.0 V5.3 File Storage; ASVS 5.0.0 V5.4 File Download.

Weakness: CWE-552.

Further reading: File Upload Cheat Sheet.

HIGH F-051 **Ethereum wallet recovery phrase hard-coded in server source**

Security · SEC-10 · effort S

RECOMMENDATION

Treat the wallet as compromised: move any assets out and stop using it. Don't keep mnemonics in code. If a comparison is needed, store only the expected public address and verify a signature instead.

The 12-word recovery phrase for a crypto wallet is written in the code. Anyone who reads the code can take everything in that wallet.

LIKELIHOOD

The phrase is in the public repository; anyone can import it into a wallet.

IMPACT

Whoever has the repository fully controls that wallet and any funds or NFTs held by its derived addresses.

DETAILS

`checkKeys` derives a wallet from the literal 12-word BIP-39 mnemonic in source in order to compare a submitted private key. Gitleaks did not flag it (not among F-001..F-008). A mnemonic yields every private key of the wallet, and endpoint responses also confirm when a submitted key matches.

EVIDENCE

routes/checkKeys.ts · lines 9–16

```
  9       const { HDNodeWallet } = await import('ethers')
 10       const mnemonic = 'purpose betray marriage blame crunch monitor spin slide
donate sport lift clutch'
 11       const mnemonicWallet = HDNodeWallet.fromPhrase(mnemonic)
 12       const privateKey = mnemonicWallet.privateKey
 13       const publicKey = mnemonicWallet.publicKey
 14       const address = mnemonicWallet.address
 15       if (req.body.privateKey === privateKey) {
 16         res.status(200).json({ success: true, message: 'Challenge successfully
solved', status: challenges.che9198e01 })
```

REFERENCES

Relevant to: A07:2025 Authentication Failures; ASVS 5.0.0 V13.3 Secret Management.

Weakness: CWE-798.

Further reading: Secrets Management Cheat Sheet.

HIGH F-052 **whoami supports JSONP with cookie auth and arbitrary fields, leaking secrets cross-site**

Security · SEC-06 · effort S

RECOMMENDATION

Remove JSONP support. Restrict `fields` to an allow-list of non-sensitive attributes (id, email, profileImage). Never keep password or TOTP data in the session cache. Set SameSite on the auth cookie.

A malicious website can quietly ask the shop "who is logged in here?" on a visitor's behalf and receive that visitor's private account details, including their scrambled password and two-factor secret.

LIKELIHOOD

Any website a logged-in shopper visits can include a script tag pointing at this endpoint.

IMPACT

The attacker's page receives the victim's email, MD5 password hash, TOTP secret and deluxe token, enough to crack the password and clone the second factor.

DETAILS

`retrieveLoggedInUser` authenticates via `req.cookies.token`, which browsers attach to cross-site script loads since no SameSite is set. It honours `?callback=`, responding with `res.jsonp`, which is designed to be readable cross-origin. The `fields` parameter copies any property of the cached user object into the response, including `password` and `totpSecret`. So `<script src="https://shop/rest/user/whoami?callback=steal&fields=email,password,totpSecret">` exfiltrates them.

EVIDENCE

routes/currentUser.ts · lines 17–33

```
17        if (security.verify(req.cookies.token)) {
18          user = security.authenticatedUsers.get(req.cookies.token)
19
20          // Parse the fields parameter into an array, splitting by comma.
21          // If not provided, both these variables will be undefined.
22          const fieldsParam = req.query?.fields as string | undefined
23          const requestedFields = fieldsParam ? fieldsParam.split(',').map(f =>
f.trim()) : []
24
25          let baseUser: any = {}
26
27          if (requestedFields.length > 0) {
28            // When fields are specified, return only those fields
```

5 more lines in the HTML report

routes/currentUser.ts · lines 53–57

```
53      if (req.query.callback === undefined) {
54        res.json(response)
55      } else {
56        res.jsonp(response)
57      }
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V3.5 Browser Origin Separation; ASVS 5.0.0 V3.4 Browser Security Mechanism Headers.

Weakness: CWE-359.

Further reading: Cross-Site Request Forgery Prevention Cheat Sheet.

HIGH F-056 **Untrusted history, usernames and reviews reach the prompt unseparated**

LLM integrations · LLM-01 · effort M

RECOMMENDATION

Keep conversation history on the server (keyed by conversation id) or at least validate that only `user`/`assistant` text roles are present, and drop client-supplied tool or system messages. Take the username out of the system prompt, or pass it as quoted data with strict character limits. Return only the needed review fields, wrapped as clearly delimited data, and require authentication to post reviews. Enforce every policy in tool code rather than the prompt, and keep sensitive tools out of conversations that have pulled in third-party content.

The chatbot cannot tell the shop's own instructions apart from text written by visitors. A visitor can plant instructions in a product review or their username, or slip fake turns into the conversation, and the chatbot may obey them, including when it talks to other customers.

LIKELIHOOD

Likely: product reviews can be written without signing in, users choose their own usernames, and the client sends the whole conversation.

IMPACT

Attackers can override the bot's rules for everyone. Planted reviews can steer other customers' chats, for example toward phishing text or tool calls made with the victim's identity.

DETAILS

Three untrusted channels reach the model with nothing separating them from instructions. (1) `messages` is taken straight from `req.body` (routes/chat.ts:187, 202). The client can send any roles, including `system` or fabricated `assistant` and tool turns, and the server never rebuilds the history it actually produced. (2) The username from the (unverified) token is placed inside the system prompt (line 82), and usernames are user-editable, so instructions planted there carry system-level authority. (3) `getProductReviews` returns full review documents (line 148). Reviews are written via PUT /rest/products/:id/reviews with no authentication and arbitrary `message`/`author` (createProductReviews.ts:19-25), so any visitor can plant indirect prompt injection that runs inside other customers' chats, where the tools act as those customers (order lookup, coupon creation, F-053). The defences are rules in the prompt only (lines 87-104). Root causes for the tools themselves are filed separately (F-053, F-054).

EVIDENCE

routes/chat.ts · lines 81–85

```
export function buildSystemPrompt (userName?: string) {
  const userIdentifier = userName ? `\nThe customer you are currently chatting with is ${userName}.` : ''
  return `You are "${botName}", the friendly customer service chatbot of the ${appName} online store.
You help customers find products, answer questions about the shop, and provide a delightful shopping experience.
Keep your responses concise and helpful.${userIdentifier}
```

routes/chat.ts · lines 141–150

```
      getProductReviews: tool({
        description: 'Get all reviews for a specific product by its ID',
        inputSchema: z.object({
          id: z.string().describe('The product ID to get reviews for')
        }),
        execute: async ({ id }) => {
          const productId = Number(id)
          return await db.reviewsCollection.find({ $where: 'this.product == ' + productId }) as Review[]
        }
      }),
```

routes/chat.ts · lines 186–203

```
    const model = config.get<string>('application.chatBot.model')
    const messages = req.body?.messages ?? []
    const userName = await getUserNameFromToken(req)

    res.setHeader('Content-Type', 'text/event-stream')
    res.setHeader('Cache-Control', 'no-cache, no-transform')
    res.setHeader('Connection', 'keep-alive')
    res.setHeader('Content-Encoding', 'identity')
    res.flushHeaders()

    const systemPrompt = buildSystemPrompt(userName)

```

6 more lines in the HTML report

routes/createProductReviews.ts · lines 14–25

```
14 export function createProductReviews () {
15   return async (req: Request, res: Response) => {
16     const user = security.authenticatedUsers.from(req)
17
18     try {
19       await reviewsCollection.insert({
20         product: req.params.id,
21         message: req.body.message,
22         author: req.body.author,
23         likesCount: 0,
24         likedBy: []
25       })
```

REFERENCES

Weakness: CWE-1427.

Further reading: LLM Prompt Injection Prevention Cheat Sheet.

MEDIUM F–014 **The application redirects to a URL specified by user-supplied input `query` that is not validated.**

Security · SEC-15 · effort S

RECOMMENDATION

Confirm the input is attacker-controlled; if so, follow the rule's references.

The application redirects to a URL specified by user-supplied input `query` that is not validated. This could redirect users to malicious locations. Consider using an allow-list approach to validate URLs, or warn users they are being redirected to a third-party website.

DETAILS

Semgrep rule rules.javascript.express.security.audit.express-open-redirect.express-open-redirect.

EVIDENCE

routes/redirect.ts · line 16

```
16       res.redirect(toUrl)
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V15.3 Defensive Coding; ASVS 5.0.0 V1.5 Safe Deserialization.

Weakness: CWE-601.

Further reading: NodeJS Security Cheat Sheet; Deserialization Cheat Sheet; Prototype Pollution Prevention Cheat Sheet.

MEDIUM F-030 **Deluxe membership granted without payment for unknown paymentMode**

Security · SEC-09 · effort S

RECOMMENDATION

Allow-list `paymentMode`, reject anything else with 400, and grant the role only after a successful charge.

The paid "deluxe" upgrade can be had for free by sending a payment type the server doesn't recognise. The server then skips payment and upgrades the account anyway.

LIKELIHOOD

Any logged-in customer can post `{"paymentMode":"x"}` directly to the API.

IMPACT

Paid membership, with its discounted prices, free delivery and lifted purchase limits, is obtained for free.

DETAILS

`upgradeToDeluxe` charges the wallet only when `paymentMode === 'wallet'` and validates a card only when `paymentMode === 'card'`. Any other value, or none at all, falls through to `user.update({ role: deluxe, ... })`. The card path also never charges the card.

EVIDENCE

routes/deluxe.ts · lines 24–47

```
24        if (req.body.paymentMode === 'wallet') {
25          const wallet = await WalletModel.findOne({ where: { UserId:
req.body.UserId } })
26          if ((wallet != null) && wallet.balance < 49) {
27            res.status(400).json({ status: 'error', error: 'Insuffienct funds in
Wallet' })
28            return
29          } else {
30            await WalletModel.decrement({ balance: 49 }, { where: { UserId:
req.body.UserId } })
31          }
32        }
33
34        if (req.body.paymentMode === 'card') {
35          const card = await CardModel.findOne({ where: { id: req.body.paymentId,
UserId: req.body.UserId } })
```

12 more lines in the HTML report

REFERENCES

Relevant to: ASVS 5.0.0 V2.2 Input Validation; ASVS v5.0.0-15.3.3 (V15.3 Defensive Coding).

Weakness: CWE-840.

Further reading: Input Validation Cheat Sheet; Mass Assignment Cheat Sheet.

MEDIUM F-044 **Cookie-authenticated profile and erasure POSTs have no CSRF defence; CORS allows all**

Security · SEC-06 · effort S

RECOMMENDATION

Set the session cookie server-side with `SameSite=Lax` (or Strict), `Secure` and `HttpOnly`. Add CSRF tokens or Origin/Referer verification on cookie-authenticated mutations. Restrict CORS to the shop's own origins.

Another website can silently make a logged-in shopper's browser change their profile or request deletion of their data, because these actions trust the login cookie alone.

LIKELIHOOD

A logged-in user only has to visit an attacker's page; the `token` cookie has no SameSite attribute set.

IMPACT

An attacker's site can change a victim's username (which also feeds the Pug injection, F-031), set their profile image URL (SSRF/CSP injection), or file a data-erasure request.

DETAILS

`POST /profile`, `POST /profile/image/url`, `POST /profile/image/file` and `POST /dataerasure` authenticate solely via `req.cookies.token` and accept form-encoded bodies (`bodyParser.urlencoded`), which cross-site HTML forms can send. There is no CSRF token, Origin check or SameSite cookie. Cookies are set by ngx-cookie on the client and by `res.cookie('token', token)` on the server, both without flags. Separately, `app.use(cors())` and `app.options('*', cors())` allow every origin on every route. That is not credentialed, but it lets any site read anonymous API responses (e.g. the user-enumerating security-question endpoint) from visitors' browsers.

EVIDENCE

routes/updateUserProfile.ts · lines 17–36

```
17      const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
18
19      if (!loggedInUser) {
20        next(new Error('Blocked illegal activity by ' +
req.socket.remoteAddress))
21        return
22      }
23
24      try {
25        const user = await UserModel.findByPk(loggedInUser.data.id)
26        if (!user) {
27          next(new Error('User not found'))
28          return
```

8 more lines in the HTML report

server.ts · lines 179–181

```
179   /* Bludgeon solution for possible CORS problems: Allow everything! */
180   app.options('*', cors())
181   app.use(cors())
```

server.ts · lines 314–318

```
314   app.use(bodyParser.urlencoded({ extended: true }))
315   /* File Upload */
316   app.post('/file-upload', uploadToMemory.single('file'), ensureFileIsPassed,
metrics.observeFileUploadMetricsMiddleware(), checkUploadSize, checkFileType,
handleZipFileUpload, handleXmlUpload, handleYamlUpload)
317   app.post('/profile/image/file', uploadToMemory.single('file'),
ensureFileIsPassed, metrics.observeFileUploadMetricsMiddleware(),
utils.asyncHandler(profileImageFileUpload()))
318   app.post('/profile/image/url', uploadToMemory.single('file'),
utils.asyncHandler(profileImageUrlUpload()))
```

routes/dataErasure.ts · lines 74–86

```
74  router.post('/', (req: Request<Record<string, unknown>, Record<string,
unknown>, DataErasureRequestParams>, res: Response, next: NextFunction): void => {
75    void (async () => {
76      const loggedInUser = security.authenticatedUsers.get(req.cookies.token)
77      if (!loggedInUser) {
78        next(new Error('Blocked illegal activity by ' +
req.socket.remoteAddress))
79        return
80      }
81
82      try {
83        await PrivacyRequestModel.create({
84          UserId: loggedInUser.data.id,
85          deletionRequested: true
```

1 more line in the HTML report

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V3.5 Browser Origin Separation; ASVS 5.0.0 V3.4 Browser Security Mechanism Headers.

Weakness: CWE-352.

Further reading: Cross-Site Request Forgery Prevention Cheat Sheet.

MEDIUM F-045 **No CSP or HSTS; profile page CSP built from a user-controlled URL**

Security · SEC-11 · effort M

RECOMMENDATION

Use `helmet()` defaults plus a strict CSP: `default-src 'self'`, no `unsafe-eval`/`unsafe-inline`, nonce-based scripts. Enable HSTS behind TLS. Never interpolate user data into headers; serve profile images only from same-origin upload paths.

The site doesn't send the standard browser instructions that limit damage from injected scripts or force encrypted connections. On the profile page a user can even rewrite those instructions themselves.

LIKELIHOOD

Applies to every page. The CSP injection needs only a logged-in user setting an image URL.

IMPACT

The XSS issues above have nothing to stop them. The one CSP present can be rewritten by the user (e.g. adding `; script-src 'unsafe-inline'`), and without HSTS tokens can leak over plain HTTP.

DETAILS

Only `helmet.noSniff()` and `helmet.frameguard()` are enabled. There is no `contentSecurityPolicy`, `hsts` or `referrerPolicy`, and `xssFilter` is commented out. On `/profile` the header is `img-src 'self' ${user.profileImage}; script-src 'self' 'unsafe-eval'`, where `profileImage` is the raw `imageUrl` saved when the server-side fetch fails (profileImageUrlUpload line 36). A value containing `;` injects arbitrary directives.

EVIDENCE

server.ts · lines 183–192

```
183   /* Security middleware */
184   app.use(helmet.noSniff())
185   app.use(helmet.frameguard())
186   // app.use(helmet.xssFilter()); // = no protection from persisted XSS via
RESTful API
187   app.disable('x-powered-by')
188   app.use(featurePolicy({
189     features: {
190       payment: ["'self'"]
191     }
192   }))
```

routes/userProfile.ts · lines 91–96

```
91       const CSP = `img-src 'self' ${user?.profileImage}; script-src 'self'
'unsafe-eval'`
92
93
94       res.set({
95         'Content-Security-Policy': CSP
96       })
```

routes/profileImageUrlUpload.ts · lines 34–36

```
34           try {
35             const user = await UserModel.findByPk(loggedInUser.data.id)
36             await user?.update({ profileImage: url })
```

REFERENCES

Relevant to: A06:2025 Insecure Design; ASVS 5.0.0 V3.4 Browser Security Mechanism Headers; ASVS 5.0.0 V12.2 HTTPS Communication with External Facing Services.

Weakness: CWE-693.

Further reading: HTTP Security Response Headers Cheat Sheet; Content Security Policy Cheat Sheet; HTTP Strict Transport Security Cheat Sheet.

MEDIUM F-047 **CAPTCHAs return their own answers and are reusable**

Security · SEC-12 · effort S

RECOMMENDATION

Return only the challenge, never the answer. Delete or mark a CAPTCHA used on first verification and fail closed when none exists. Add express-rate-limit (keyed on real client IP and user ID) to feedback, export, search, upload and login. Set `limits: { fileSize }` on every multer instance.

The "prove you're human" puzzles send the correct answer along with the question, so bots can pass them every time. Nothing else limits how often these forms can be submitted.

LIKELIHOOD

Any script can call /rest/captcha and read `answer` from the JSON.

IMPACT

Unlimited automated feedback submissions (spam, stored-XSS delivery, database growth) and unlimited data-export requests.

DETAILS

`captchas()` responds with `{ captchaId, captcha, answer }`, and `verifyCaptcha` never deletes a used CAPTCHA, so one ID/answer pair can be replayed indefinitely. `imageCaptchas()` likewise returns `answer`. `verifyImageCaptcha` calls `next()` when the user has no recent CAPTCHA at all (`!captchas[0]`). There is no rate limiter on `/api/Feedbacks`, `/rest/user/data-export`, `/rest/products/search` or `/file-upload`. The only limiters cover reset-password and 2FA, and they are spoofable (F-027). `uploadToDisk` for `/rest/memories` has no `limits.fileSize`.

EVIDENCE

routes/captcha.ts · lines 24–41

```
24      const captcha = {
25        captchaId,
26        captcha: expression,
27        answer
28      }
29      const captchaInstance = CaptchaModel.build(captcha)
30      await captchaInstance.save()
31      res.json(captcha)
32    }
33  }
34
35  export const verifyCaptcha = () => async (req: Request, res: Response, next:
NextFunction) => {
```

6 more lines in the HTML report

routes/imageCaptcha.ts · lines 24–53

```
24        const imageCaptcha = {
25          image: captcha.data,
26          answer: captcha.text,
27          UserId: user.data.id
28        }
29        const imageCaptchaInstance = ImageCaptchaModel.build(imageCaptcha)
30        await imageCaptchaInstance.save()
31        res.json(imageCaptcha)
32      } catch (error) {
33        res.status(400).send(res.__('Unable to create CAPTCHA. Please try
again.'))
34      }
35    }
```

18 more lines in the HTML report

server.ts · lines 693–712

```
693 const uploadToDisk = multer({
694   storage: multer.diskStorage({
695     destination: (req: Request, file: any, cb: any) => {
696       const isValid = mimeTypeMap[file.mimetype]
697       let error: Error | null = new Error('Invalid mime type')
698       if (isValid) {
699         error = null
700       }
701       cb(error,
path.resolve('frontend/dist/frontend/assets/public/images/uploads/'))
702     },
703     filename: (req: Request, file: any, cb: any) => {
704       const name = security.sanitizeFilename(file.originalname)
```

8 more lines in the HTML report

REFERENCES

Relevant to: ASVS v5.0.0-2.4.1 (V2.4 Anti-automation).

Weakness: CWE-804.

Further reading: Denial of Service Cheat Sheet.

MEDIUM F-049 **Development error handler returns stack traces and SQL errors to clients**

Security · SEC-13 · effort S

RECOMMENDATION

Register `errorhandler` only when `NODE_ENV === 'development'`. In production, log the error server-side with a correlation ID and return a generic message. Never forward DB error objects to the client.

When something goes wrong, the server shows the visitor its internal error details, including database errors. That gives attackers a map of the system.

LIKELIHOOD

Any malformed request reaches it, e.g. a single quote in the search box.

IMPACT

SQL error text reveals table and column structure, which speeds up the SQL injection in F-012/F-015. Stack traces reveal file paths and library versions. XXE and YAML results are echoed through the same handler.

DETAILS

`app.use(errorhandler())` is registered unconditionally. The `errorhandler` package is intended for development only and renders full stack traces as HTML/JSON. Its title is even set to include the Express version. `searchProducts` passes `error.parent` (the raw SQLite error with the query) to `next`. Many handlers build error messages containing user input or remote addresses (`'Blocked illegal activity by ' + remoteAddress`).

EVIDENCE

server.ts · lines 674–675

```
674   /* Error Handling */
675   app.use(errorhandler())
```

server.ts · line 724

```
724 errorhandler.title = `${config.get<string>('application.name')} (Express ${utils.version('express')})`
```

routes/search.ts · lines 30–32

```
30        }).catch((error: ErrorWithParent) => {
31          next(error.parent)
32        })
```

REFERENCES

Relevant to: A10:2025 Mishandling of Exceptional Conditions; ASVS 5.0.0 V16.2 General Logging; ASVS 5.0.0 V16.5 Error Handling; ASVS v5.0.0-13.4.2 (V13.4 Unintended Information Leakage).

Weakness: CWE-209.

Further reading: Logging Cheat Sheet; Error Handling Cheat Sheet.

MEDIUM F-057 **Chat stream has no timeout, no cancel on disconnect, and leaks raw errors**

LLM integrations · LLM-07 · effort S

RECOMMENDATION

Create an AbortController per request, abort it on `req.on('close')`, and pass it with `AbortSignal.timeout(...)` as `abortSignal` to `streamText`. Send clients a generic error code instead of `event.error`. Handle `length` and `content-filter` finish reasons in the UI. Consider a fallback model or a static 'assistant unavailable' answer.

If the AI provider is slow or a customer closes the chat, the server keeps the request running and paying for it, with no cut-off time. Raw technical error messages from the AI service can also be shown to customers.

LIKELIHOOD

Happens whenever the provider is slow or overloaded, or a browser tab closes mid-answer.

IMPACT

Generation keeps running and billing for clients that are gone, requests can hang indefinitely, and provider error details reach end users.

DETAILS

`createOpenAICompatible` is built without a custom `fetch` or timeout (routes/chat.ts:107-111), and `streamText` gets no `abortSignal` (lines 199-209). Nothing listens for `req.on('close')`, so when the client disconnects the multi-step tool loop (up to 10 steps, plus retries) runs to the end. Retries rely on the SDK's default backoff, and there is no fallback model or cached answer. For stream `error` events, the raw error is interpolated into the SSE payload sent to the browser (line 249), even though `summarizeLlmError` exists and is used only for logs. A `finishReason` of `length` or `content-filter` is forwarded but the frontend ignores it (chat.service.ts:68-77), so truncated answers look complete to the customer.

EVIDENCE

routes/chat.ts · lines 107–111

```
107 const provider = createOpenAICompatible({
108   name: 'juice-shop-llm',
109   apiKey: process.env.LLM_API_KEY ?? '',
110   baseURL: config.get<string>('application.chatBot.llmApiUrl')
111 })
```

routes/chat.ts · lines 199–211

```
199       const result = streamText({
200         model: provider(model),
201         system: systemPrompt,
202         messages,
203         tools: { ...chatTools },
204         maxRetries: config.get<number>('application.chatBot.llmMaxRetries'),
205         stopWhen: stepCountIs(10),
206         onError: ({ error }) => {
207           logger.warn('Chatbot stream error: ' + summarizeLlmError(error))
208         }
209       })
210
```

1 more line in the HTML report

routes/chat.ts · lines 248–250

```
248           case 'error':
249             res.write(`data: ${JSON.stringify({ error: `LLM error:
${event.error as string}` })}\n\n`)
250             break
```

frontend/src/app/Services/chat.service.ts · lines 68–77

```
68              const delta = parsed.choices?.[0]?.delta
69              const finishReason = parsed.choices?.[0]?.finish_reason
70
71              if (delta) {
72                const chunk: ChatChunk = {}
73                if (delta.content) chunk.deltaContent = delta.content
74                if (delta.tool_calls) chunk.deltaToolCalls = delta.tool_calls
75                if (finishReason) chunk.finishReason = finishReason
76                if (chunk.deltaContent || chunk.deltaToolCalls ||
chunk.finishReason) {
77                  chunks.push(chunk)
```

REFERENCES

Weakness: CWE-400.

LOW F-041 **Metrics, full app configuration and API docs served publicly**

Security · SEC-03 · effort S

RECOMMENDATION

Protect /metrics with a bearer token or bind it to an internal port. Return an explicit allow-list of client-needed settings instead of the whole config. Put admin routes behind an admin role check.

Internal statistics about the shop (how many customers, orders and wallet money) and its full settings are visible to anyone on the internet without logging in.

LIKELIHOOD

Anyone can fetch /metrics, /rest/admin/application-configuration and /api-docs.

IMPACT

Business figures (user counts, order totals, total wallet balance), process internals and the complete runtime configuration help attackers plan, and leak commercially sensitive numbers.

DETAILS

`/metrics` is registered twice with no auth and exposes Prometheus gauges including `wallet_balance_total`, user and order counts, and Node process metrics. `retrieveAppConfiguration` returns `config.util.toObject(config)` with only `chatBot.llmApiUrl` removed, so any secret later added to config ships to anonymous callers. The route sits under `/rest/admin/` but has no guard.

EVIDENCE

server.ts · line 670

```
670   app.get('/metrics', utils.asyncHandler(metrics.serveMetrics()))
```

server.ts · line 723

```
723 app.get('/metrics', utils.asyncHandler(metrics.serveMetrics()))
```

routes/appConfiguration.ts · lines 9–17

```
 9 export function retrieveAppConfiguration () {
10   return (_req: Request, res: Response) => {
11     const safeConfig = structuredClone(config.util.toObject(config))
12     if (safeConfig.application?.chatBot) {
13       delete safeConfig.application.chatBot.llmApiUrl
14     }
15     res.json({ config: safeConfig })
16   }
17 }
```

REFERENCES

Relevant to: A01:2025 Broken Access Control; ASVS 5.0.0 V8.2 General Authorization Design; ASVS 5.0.0 V8.3 Operation Level Authorization; ASVS 5.0.0 V14.2 General Data Protection.

Weakness: CWE-200.

Further reading: Authorization Cheat Sheet; Insecure Direct Object Reference Prevention Cheat Sheet.

LOW F–059 **No tests or evals for the chatbot prompt and tool policies**

LLM integrations · LLM-08 · effort M

RECOMMENDATION

Add an eval suite (against a mock provider plus scheduled runs against the real model) covering coupon refusal, cross-user order requests and injected reviews. Log one structured line per call with model, user id, token counts, tools called and finish reason. Add a thumbs-down control in the chat UI.

Nothing automatically checks that the chatbot follows the shop's rules after the prompt or AI model changes, and there is no way to trace a bad answer back to a specific conversation.

LIKELIHOOD

Any change to the prompt or model (for example a new `gemma` tag) can silently change coupon and order behaviour.

IMPACT

Regressions in policy compliance or answer quality ship unnoticed, and bad answers cannot be traced to a user or conversation.

DETAILS

The repository has no tests that cover `/rest/chat`, `buildSystemPrompt` or the tools (no matches under test/). Observability is limited to global Prometheus counters for tokens and tool calls (routes/chat.ts:57-79) with no per-request record of model, user, tool arguments or finish reason. The UI has no way to flag a bad answer. The prompt is versioned in code, which is good.

EVIDENCE

routes/chat.ts · lines 57–79

```
57 const metricInputTokensTotal = new Counter({
58   name: `${app}_llm_input_tokens_total`,
59   help: 'Number of total input tokens processed',
60 })
61 const metricInputTokens = new Counter({
62   name: `${app}_llm_input_tokens`,
63   help: 'Number of input tokens processed',
64   labelNames: ['type'],
65 })
66 const metricOutputTokensTotal = new Counter({
67   name: `${app}_llm_output_tokens_total`,
68   help: 'Number of total output tokens processed',
```

11 more lines in the HTML report

routes/chat.ts · lines 81–105

```
81 export function buildSystemPrompt (userName?: string) {
82   const userIdentifier = userName ? `\nThe customer you are currently chatting with is ${userName}.` : ''
83   return `You are "${botName}", the friendly customer service chatbot of the ${appName} online store.
84 You help customers find products, answer questions about the shop, and provide a delightful shopping experience.
85 Keep your responses concise and helpful.${userIdentifier}
86
87 IMPORTANT RULES:
88 - You MUST use the searchProducts tool whenever a customer asks about products, availability, prices, or anything related to the shop's catalog. NEVER guess or make up product names, prices, or descriptions.
89 - You MUST use the getProductReviews tool whenever a customer asks for reviews of a product.
90 - You MUST use the getOrderById tool whenever a customer asks about a specific order by its ID.
91 - Only recommend or mention products that were returned by the searchProducts tool. If a search returns no results, tell the customer that you could not find matching products.
92 - Do NOT invent information. If you do not know the answer to a question, say so honestly.
```

13 more lines in the HTML report

Open questions

Points the code alone could not settle; each needs an answer from the team.

QUESTION F-058 **Which LLM provider runs in production, and with what data retention?**

LLM integrations · LLM-05 · effort S

RECOMMENDATION

Confirm the production provider, that the connection uses HTTPS, and that zero-retention / no-training terms are in place. Minimise the order and review fields passed to the model, and record the provider in the privacy policy.

The chatbot sends customer names, order details and conversation text to whichever AI service is configured. We need to know which service that is in production, and whether it keeps or trains on that data.

LIKELIHOOD

Unknown: the endpoint is set by configuration (default: a local Ollama on localhost).

IMPACT

If a third-party API is used, customer names, order contents and reviews go to that provider under its retention and training terms.

DETAILS

The prompt contains the customer's username (routes/chat.ts:82), full order documents including masked email, products, address and payment IDs (line 170), and full review documents including author emails (line 148). `llmApiUrl` comes from config (default `http://localhost:11434/v1`, config/default.yml:18-23) and may be pointed at a hosted provider in deployment. Prompts and responses are not logged in full (only summarised errors), which is good.

EVIDENCE

config/default.yml · lines 18–23

```
18   chatBot:
19     name: 'Juicy the Smart Assistant'
20     avatar: 'JuicyChatBot.png'
21     model: 'gemma4:e4b'
22     llmApiUrl: 'http://localhost:11434/v1'
23     llmMaxRetries: 2
```

routes/chat.ts · lines 165–170

```
165           const order = await db.ordersCollection.findOne({ orderId })
166
167           if (!order) return { error: 'Order not found' }
168           if (order.email !== maskedEmail) return { error: 'Order does not
belong to the current customer' }
169
170           return order
```

REFERENCES

Relevant to: ASVS 5.0.0 V14.2 General Data Protection.

Technical details

- Models that served calls: claude-opus-5-5.
- Scanners: ghcr.io/google/osv-scanner@sha256:afd838850ac1a0fcc15ff4a041dc9ba11123c3f0d2666217a5f0fcf9222b55fa, semgrep/semgrep@sha256:93963d9295a366f59e4850127b1550400ee7b388f04fe144e4a1f6325d96e01b, ghcr.io/gitleaks/gitleaks@sha256:c00b6bd0aeb3071cbcb79009cb16a60dd9e0a7c60e2be9ab65d25e6bc8abbb7f. Semgrep rulesets: javascript (74 rules, sha256 ccd47b2aeb2f), typescript (74 rules, sha256 6248ea7477e6), react (4 rules, sha256 a5c18c389e94). OSV queried 2026-10-07.
- Budgets are checked between model calls; a call in progress may exceed its share by its own cost.

Disclaimer

An audit finds issues; it does not certify their absence. Findings describe the code at the commits listed above.

Andrii Naidenko · 2026-10-11